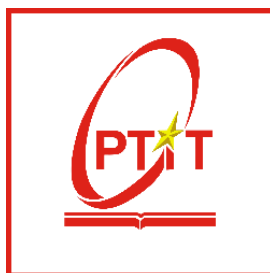


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA HỆ THỐNG THÔNG TIN



BÁO CÁO CUỐI KỲ
CÁC HỆ THỐNG PHÂN TÁN

ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG CHAT NGANG HÀNG P2P

- Nhóm 16 -

Giảng viên hướng dẫn: Kim Ngọc Bách
Sinh viên thực hiện: Hoàng Minh Nghĩa
Mã sinh viên: B22DCCN605

Hà Nội - 2026

MỤC LỤC

MỤC LỤC	
DANH MỤC HÌNH ẢNH.....	
LỜI MỞ ĐẦU	1
CHƯƠNG I: TỔNG QUAN ĐỀ TÀI.....	2
1.1. GIỚI THIỆU BÀI TOÁN	2
1.2. MỤC TIÊU NGHIÊN CỨU	3
1.3. PHẠM VI NGHIÊN CỨU	4
1.4. TỔNG KẾT CHƯƠNG	7
CHƯƠNG II: CƠ SỞ LÝ THUYẾT	8
2.1. MÔ HÌNH MẠNG NGANG HÀNG PEER-TO-PEER.....	8
2.2. GIAO THỨC ĐIỀU KHIỂN TRUYỀN VẬN TCP VÀ LẬP TRÌNH SOCKET.....	10
2.3. LẬP TRÌNH ĐA LUỒNG VÀ CƠ CHẾ ĐỒNG BỘ HÓA	13
2.4. MẬT MÃ HỌC VÀ GIAO THỨC BẢO MẬT ĐẦU CUỐI.....	15
2.5. TUẦN TỰ HÓA DỮ LIỆU VÀ MÃ HÓA BASE64	17
2.6. LẬP TRÌNH GIAO DIỆN HƯỚNG SỰ KIỆN VỚI TKINTER.....	18
2.7. TỔNG KẾT CHƯƠNG	20
CHƯƠNG III: XÂY DỰNG HỆ THỐNG	21
3.1. KIẾN TRÚC TỔNG THỂ HỆ THỐNG.....	21
3.2. CẤU TRÚC GIAO THỨC TRAO ĐỔI THÔNG điệp	23
3.3. CÁC CƠ CHẾ LỖI CỦA HỆ THỐNG.....	26
3.4. THIẾT KẾ KIẾN TRÚC GIAO DIỆN NGƯỜI DÙNG ĐA LUỒNG	31
3.5. TỔNG KẾT CHƯƠNG	32
CHƯƠNG IV: THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG.....	34
4.1. MÔI TRƯỜNG THỰC NGHIỆM.....	34
4.2. KỊCH BẢN THỰC NGHIỆM CHỨC NĂNG.....	35
4.3. MÔ PHỎNG BIẾN ĐỘNG MẠNG VỚI CHURN SIMULATOR	43
4.4. ĐÁNH GIÁ HỆ THỐNG.....	45
4.5. ĐỊNH HƯỚNG PHÁT TRIỂN TƯƠNG LAI	46
LỜI KẾT.....	47
TÀI LIỆU THAM KHẢO	48

DANH MỤC HÌNH ẢNH

HÌNH 2.1: SƠ ĐỒ SO SÁNH HAI MÔ HÌNH KIẾN TRÚC P2P	10
HÌNH 3.1: SƠ ĐỒ KIẾN TRÚC TỔNG QUAN HỆ THỐNG	21
HÌNH 3.2: CẤU TRÚC MỘT GÓI TIN NHẮN ĐƯỢC ĐÓNG GÓI	24
HÌNH 3.3: SƠ ĐỒ TUẦN TỰ CƠ CHẾ CHUYỂN TIẾP OFFLINE.....	28
HÌNH 3.4: SƠ ĐỒ TUẦN TỰ TRUYỀN TẬP TIN BA BƯỚC	30
HÌNH 4.1: GIAO DIỆN ĐĂNG NHẬP HỆ THỐNG.....	35
HÌNH 4.2: GIAO DIỆN CHÍNH CỦA HỆ THỐNG	36
HÌNH 4.3: B NHẬN ĐƯỢC TIN NHẮN TỪ A.....	37
HÌNH 4.4: KHÔNG CÓ GÓI TIN CHỨA NỘI DUNG TIN NHẮN TRÊN TRACKER SERVER	37
HÌNH 4.5: GIAO DIỆN TẠO NHÓM	38
HÌNH 4.6: KHI NHẤN VÀO NHÓM MUỐN GỬI TIN NHẮN, TÊN NGƯỜI NHẬN ĐƯỢC TỰ ĐỘNG ĐIỀN	38
HÌNH 4.7: TIN NHẮN NHÓM TỪ A HIỂN THỊ TRÊN GIAO DIỆN CỦA C.....	39
HÌNH 4.8: GIAO DIỆN B XUẤT HIỆN THÔNG BÁO SAU KHI A GỬI FILE CHO B.....	40
HÌNH 4.9: Ở KỊCH BẢN 1, THÔNG BÁO B TỪ CHỐI NHẬN FILE Ở GIAO DIỆN CỦA A	41
HÌNH 4.10: Ở KỊCH BẢN 2, THÔNG BÁO B ĐỒNG Ý NHẬN FILE Ở GIAO DIỆN CỦA A	41
HÌNH 4.11: TIN NHẮN TỰ ĐỘNG ĐƯỢC LƯU TRỮ LÊN TRACKER SERVER .	42
HÌNH 4.12: THÔNG BÁO LƯU TẠM TIN NHẮN TRÊN TRACKER SERVER	42
HÌNH 4.13: TIN NHẮN ĐƯỢC CHUYỂN ĐẾN B NGAY KHI B ONLINE TRỞ LẠI	43
HÌNH 4.14: KHỞI CHẠY CHURN_SIMULATE.PY	44
HÌNH 4.15: HỆ THỐNG VẪN HOẠT ĐỘNG BÌNH THƯỜNG DÙ CHO NHIỀU BOT RA VÀO MẠNG LIÊN TỤC	44

LỜI MỞ ĐẦU

Trong kỷ nguyên công nghệ số hiện nay, các ứng dụng nhắn tin và trao đổi thông tin trực tuyến đã trở thành một phần không thể thiếu trong đời sống cá nhân cũng như công việc. Hầu hết các ứng dụng phổ biến trên thị trường hiện nay đều được xây dựng dựa trên mô hình Client - Server truyền thống. Mặc dù mô hình này mang lại khả năng quản lý tập trung và tốc độ xử lý nhanh, nhưng nó cũng tồn tại những hạn chế nhất định như: nguy cơ rò rỉ dữ liệu cá nhân tại máy chủ trung tâm, bottleneck khi lưu lượng mạng quá lớn, và rủi ro toàn bộ hệ thống tê liệt khi máy chủ gặp sự cố.

Để giải quyết những vấn đề trên, mô hình mạng ngang hàng Peer-to-Peer (P2P) nổi lên như một giải pháp thay thế hiệu quả. Trong mạng P2P, mỗi máy tính tham gia mạng vừa đóng vai trò là máy khách vừa là máy chủ, giúp phân tán tải trọng mạng, loại bỏ sự phụ thuộc hoàn toàn vào máy chủ trung tâm và nâng cao tính bảo mật, quyền riêng tư cho người dùng.

Nhận thức được tầm quan trọng và tính ứng dụng thực tiễn của công nghệ này, em/nhóm chúng em đã quyết định chọn đề tài "Phát triển hệ thống chat ngang hàng P2P" làm đề án kết thúc học phần.

Mục tiêu chính của đề án là ứng dụng các kiến thức về lập trình mạng, TCP Socket, và xử lý đa luồng bằng ngôn ngữ Python để xây dựng thành công một phần mềm chat P2P hoàn chỉnh. Hệ thống không chỉ đáp ứng các tính năng cơ bản như nhắn tin trực tiếp, chat nhóm, mà còn tích hợp các cơ chế nâng cao như: Mã hóa đầu cuối, truyền tải file dữ liệu, cơ chế Store-and-Forward xử lý tin nhắn ngoại tuyến và một giao diện trực quan thân thiện với người dùng.

Mặc dù đã nỗ lực hết sức, nhưng do giới hạn về thời gian và kiến thức, hệ thống cũng như bản báo cáo chắc chắn không tránh khỏi những thiếu sót. Vì vậy, nhóm em rất mong nhận được sự góp ý, chỉ bảo tận tình từ Thầy để đề án ngày càng hoàn thiện hơn, xin trân trọng cảm ơn!

CHƯƠNG I: TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu bài toán

Trong những thập kỷ gần đây, sự bùng nổ của Internet đã thúc đẩy sự phát triển mạnh mẽ của các ứng dụng truyền thông và mạng xã hội. Các hệ thống nhắn tin trực tuyến hiện đại phần lớn được xây dựng dựa trên kiến trúc Client - Server. Trong mô hình này, tất cả dữ liệu, thông điệp và tệp tin của người dùng đều phải đi qua một hoặc một cụm máy chủ trung tâm để xử lý trước khi được phân phối đến người nhận.

Mặc dù mô hình Client - Server mang lại sự thuận tiện trong việc quản lý và đồng bộ dữ liệu, nhưng đối với các hệ thống yêu cầu tính riêng tư cao và hoạt động liên tục, nó bộc lộ một số nhược điểm chí mạng trong kiến trúc hệ thống phân tán:

- Điểm lỗi rủi ro duy nhất (Single Point of Failure): Nếu máy chủ trung tâm gặp sự cố do quá tải, lỗi phần cứng, hoặc bị tấn công từ chối dịch vụ DDoS, toàn bộ mạng lưới giao tiếp của người dùng sẽ bị tê liệt hoàn toàn.
- Nút thắt cổ chai về hiệu suất (Bottleneck): Khi số lượng người dùng tăng vọt, máy chủ phải chịu tải trọng khổng lồ để duy trì hàng triệu kết nối đồng thời và chuyển tiếp băng thông, đặc biệt là khi truyền file, đòi hỏi chi phí vận hành và mở rộng hạ tầng rất tốn kém.
- Rủi ro về quyền riêng tư và bảo mật: Toàn bộ thông điệp giao tiếp đều đi qua và lưu trữ trên máy chủ. Nếu máy chủ bị tấn công hoặc xâm nhập, dữ liệu nhạy cảm của người dùng có nguy cơ bị rò rỉ hàng loạt.

Để giải quyết các bài toán trên, kiến trúc mạng ngang hàng Peer-to-Peer (P2P) được đề xuất như một mô hình thay thế ưu việt. Trong mạng P2P, không có sự phân biệt rạch ròi giữa máy khách và máy chủ. Mỗi nút tham gia mạng đều có quyền bình đẳng, vừa đóng vai trò là máy khách (yêu cầu tài nguyên), vừa là máy chủ (cung cấp tài nguyên). Do đó, dữ liệu được truyền tải trực tiếp giữa các thiết bị với nhau mà không cần đi qua nút trung gian.

Để áp dụng và chứng minh những ưu điểm lý thuyết của kiến trúc phân tán này vào thực tiễn, đề tài "**Phát triển hệ thống chat ngang hàng P2P**" đã được lựa chọn để nghiên cứu và triển khai trong khuôn khổ đồ án môn học.

1.2. Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đồ án không chỉ dừng lại ở việc lập trình một ứng dụng thông thường, mà là quá trình nghiên cứu, ứng dụng và kiểm chứng các nguyên lý nền tảng của hệ thống phân tán, giao tiếp liên tiến trình qua mạng và kỹ thuật lập trình đa luồng.

Để minh chứng cho việc nắm vững các lý thuyết trên, đồ án đặt ra và hướng tới hoàn thành các mục tiêu chi tiết sau đây:

- **Xây dựng và làm chủ kiến trúc mạng ngang hàng lai (Hybrid P2P):** Thay vì sử dụng mạng P2P thuần túy vốn gặp khó khăn trong việc tìm kiếm thiết bị ban đầu, đồ án hướng tới việc xây dựng một hệ thống Hybrid P2P tối ưu hơn. Trong đó, hệ thống bao gồm một máy chủ dẫn đường - Tracker Server đóng vai trò nhẹ chỉ để quản lý danh bạ mạng, và các nút ngang hàng - Peer Node có khả năng tự thiết lập kết nối TCP trực tiếp với nhau. Mục tiêu là đảm bảo luồng dữ liệu như tin nhắn, file hoàn toàn không chạy qua máy chủ trung tâm, giảm tải tối đa cho Tracker.
- **Triển khai các cơ chế giao tiếp và đồng bộ cơ bản:** Xây dựng thuật toán giúp các nút mới gia nhập mạng có thể ngay lập tức phát hiện và đồng bộ danh bạ với các nút đang trực tuyến khác thông qua Tracker.

Định tuyến tin nhắn: Hiện thực hóa giao thức truyền tin nhắn trực tiếp 1-1 và tin nhắn nhóm bằng cách mô phỏng Broadcast qua Unicast tuần tự, đảm bảo thông điệp đi đến đúng địa chỉ đích thông qua các socket độc lập.

- **Phát triển các tính năng nâng cao nhằm tối ưu hóa hệ thống:**
 - **Truyền tệp tin an toàn:** Xây dựng cơ chế bắt tay 3 bước ở tầng ứng dụng bao gồm: Yêu cầu -> Xác nhận đồng ý -> Truyền dữ liệu. Điều này giúp hệ thống hoạt động tương tự các ứng dụng thực tế, tôn trọng quyền từ chối nhận file của người dùng đích.

- Lưu trữ và chuyển tiếp: Giải quyết bài toán nút đích bị mất kết nối bằng cách biến Tracker Server thành một "hộp thư tạm thời". Tin nhắn sẽ được gửi lên lưu trữ ngầm tại Tracker và tự động đẩy về ngay khi nút đích kết nối trở lại.
- Bảo mật đầu cuối: Áp dụng thuật toán mã hóa đối xứng để băm nội dung thông điệp trước khi đưa ra đường truyền mạng TCP, đảm bảo ngăn chặn các cuộc tấn công nghe lén và bảo vệ dữ liệu ngay cả khi lưu trữ tạm thời trên Tracker Server.
- Thiết kế giao diện người dùng đa luồng: Mục tiêu là giải quyết bài toán "đóng băng giao diện" thường gặp trong lập trình ứng dụng mạng. Thông qua việc sử dụng cấu trúc dữ liệu hàng đợi (Queue) để tách biệt hoàn toàn luồng giao diện và các luồng mạng, hệ thống phải đảm bảo trải nghiệm người dùng mượt mà theo thời gian thực.
- Đảm bảo tính sẵn sàng và khả năng chịu lỗi: Hệ thống phân tán luôn phải đối mặt với sự không ổn định của mạng. Do đó, mục tiêu quan trọng là thiết lập cơ chế Heartbeat để liên tục kiểm tra trạng thái sống của các nút. Đồng thời, hệ thống phải vượt qua được bài kiểm tra "Churn Simulation" - mô phỏng kịch bản khắc nghiệt khi hàng loạt các nút gia nhập và thoát khỏi mạng lưới một cách ngẫu nhiên và liên tục, chứng minh Tracker Server không bị tràn bộ nhớ hay sập hệ thống.

1.3. Phạm vi nghiên cứu

1.3.1. Phạm vi công nghệ và kiến trúc phần mềm

Việc lựa chọn công cụ và thiết kế kiến trúc đóng vai trò quyết định đến hiệu năng và tính ổn định của hệ thống. Trong phạm vi đề án này, hệ sinh thái công nghệ được lựa chọn với tiêu chí tối ưu hóa khả năng giao tiếp mạng và đơn giản hóa quá trình phát triển, cụ thể bao gồm:

- Ngôn ngữ lập trình nền tảng: Đề án lựa chọn Python làm ngôn ngữ phát triển. Mặc dù Python tồn tại cơ chế Global Interpreter Lock hạn chế khả năng tính toán song song thực sự của CPU, nhưng đối với các ứng dụng nặng về giao tiếp mạng

như hệ thống chat, cơ chế đa luồng của Python vẫn đáp ứng xuất sắc việc tối ưu hóa thời gian chờ phản hồi từ socket mà không làm giảm hiệu suất tổng thể.

- Giao thức mạng ở tầng giao vận: Đề án giới hạn ở việc sử dụng duy nhất giao thức Transmission Control Protocol (TCP). Không giống như các hệ thống gọi thoại hoặc video streaming thường dùng UDP để ưu tiên tốc độ, một hệ thống chat và truyền tải tệp tin bắt buộc phải đảm bảo tính toàn vẹn của dữ liệu và thứ tự gói tin. TCP cung cấp sẵn cơ chế bắt tay 3 bước, kiểm soát luồng và truyền lại gói tin lỗi, giúp giảm bớt gánh nặng xử lý các lỗi vật lý ở tầng ứng dụng.
- Định dạng dữ liệu giao tiếp: Toàn bộ thông điệp giao tiếp giữa Peer-Tracker và Peer-Peer được chuẩn hóa dưới dạng cấu trúc JSON. Để khắc phục đặc điểm stream-based của TCP vốn dễ gây ra lỗi phân mảnh hoặc dính gói tin, hệ thống áp dụng kỹ thuật ngắt gói bằng ký tự đặc biệt, giúp các nút có thể dễ dàng tách và phân tích từng gói tin một cách độc lập.
- Xử lý đồng thời và kiến trúc giao diện: Đề án áp dụng mẫu thiết kế Producer-Consumer (Người sản xuất - Người tiêu thụ) thông qua cấu trúc dữ liệu queue.Queue an toàn luồng của Python. Giao diện người dùng được xây dựng trên tkinter sẽ hoạt động như một Consumer liên tục lấy dữ liệu từ Queue, trong khi các luồng mạng hoạt động như Producer. Kiến trúc này giới hạn sự can thiệp trực tiếp của luồng mạng vào giao diện, ngăn ngừa tuyệt đối lỗi crash ứng dụng do xung đột luồng.

1.3.2. Phạm vi môi trường triển khai và cấu trúc liên kết mạng

Cấu trúc liên kết mạng và môi trường triển khai ảnh hưởng trực tiếp đến cách thức các nút mạng tìm kiếm và giao tiếp với nhau. Đề án giới hạn không gian hoạt động để tập trung vào logic định tuyến nội bộ, tránh các vấn đề hạ tầng mạng rườm rà. Chi tiết như sau:

- Môi trường vận hành cục bộ: Hệ thống được thiết kế và tối ưu để chạy kiểm thử trong môi trường mạng cục bộ LAN/WLAN hoặc chạy giả lập trên cùng một máy vật lý (Localhost) thông qua kỹ thuật phân bổ và liên kết cổng độc lập cho từng tiến trình.

- Giới hạn về dịch địa chỉ mạng: Đồ án tập trung nghiên cứu bản chất định tuyến và thuật toán giao tiếp P2P. Việc triển khai hệ thống trên môi trường Internet diện rộng thường vấp phải rào cản từ các bộ định tuyến Network Address Translation (NAT) và tường lửa của nhà mạng, ngăn chặn các kết nối TCP trực tiếp từ bên ngoài vào. Các kỹ thuật phức tạp để đục lỗ NAT hay sử dụng máy chủ hỗ trợ STUN/TURN/ICE sẽ nằm ngoài phạm vi nghiên cứu của đề tài này nhằm giữ cho kiến trúc hệ thống gọn nhẹ và tập trung đúng vào trọng tâm học thuật.

1.3.3. Phạm vi quản trị dữ liệu và xử lý tệp tin

Trong hệ thống phân tán, chiến lược lưu trữ trạng thái mạng và truyền tải dữ liệu dung lượng lớn luôn đòi hỏi sự đánh đổi giữa hiệu suất và độ phức tạp. Các giới hạn về quản lý dữ liệu trong hệ thống được thiết lập như sau:

- Lưu trữ dữ liệu trên bộ nhớ chính: Hệ thống không tích hợp các hệ quản trị cơ sở dữ liệu như MySQL, PostgreSQL, hay SQLite. Toàn bộ trạng thái mạng bao gồm danh bạ các nút trực tuyến, danh sách các nhóm và hộp thư tạm đều được Tracker Server quản lý trực tiếp trên RAM thông qua các cấu trúc dữ liệu cơ sở. Sự đánh đổi này mang lại tốc độ truy xuất cực nhanh và giảm thiểu độ phức tạp cài đặt, đồng thời dữ liệu sẽ bị bay hơi khi Server khởi động lại.
- Cơ chế đóng gói truyền file bằng Base64: Để tận dụng chung một luồng kết nối socket bằng JSON cho cả văn bản và tệp tin, đồ án sử dụng kỹ thuật mã hóa tệp nhị phân sang định dạng văn bản Base64 Encoding. Phương pháp này gia tăng khoảng 33% dung lượng truyền tải và yêu cầu nạp toàn bộ tệp vào RAM trước khi gửi, do đó chỉ phù hợp trong phạm vi truyền các tệp tin có kích thước nhỏ gọn như văn bản, hình ảnh. Kỹ thuật cắt nhỏ tệp tin qua một kênh socket nhị phân riêng biệt được xác định là nằm ngoài giới hạn của phiên bản này.
- Mô hình mã hóa tĩnh: Mặc dù hệ thống có áp dụng mã hóa đầu cuối bằng thuật toán mã hóa đối xứng AES thông qua thư viện cryptography module Fernet, khóa bí mật được giới hạn ở dạng "khóa chia sẻ trước" cấu hình tĩnh trong mã nguồn. Việc xây dựng cơ sở hạ tầng khóa công khai và thuật toán trao đổi khóa động như RSA hoặc Diffie-Hellman Key Exchange giữa các peer không nằm trong phạm vi thực hiện của đồ án.

1.3.4. Phạm vi thử nghiệm và đánh giá

Quy trình thử nghiệm được thiết kế nhằm phản ánh chính xác độ tin cậy và khả năng chịu lỗi của hệ thống phân tán thay vì đo lường giới hạn sức mạnh phần cứng tuyệt đối. Các tiêu chí đánh giá được khoanh vùng bao gồm:

- Mức độ đánh giá hiệu năng hệ thống không tập trung vào các chỉ số như băng thông tối đa hay độ trễ tính bằng mili-giây do hệ thống chạy trên môi trường cục bộ.
- Thay vào đó, phạm vi thử nghiệm giới hạn ở việc kiểm thử tính ổn định của thuật toán đồng bộ trạng thái và độ bền bỉ của Tracker Server. Thông qua kịch bản mô phỏng biến động mạng Churn Simulation, hệ thống sẽ được đánh giá khả năng bất ngờ ngoại lệ, xử lý lỗi giải phóng tài nguyên và tính năng tự động cập nhật danh bạ khi phải đối mặt với tình trạng hàng chục thiết bị gia nhập và thoát mạng bất ngờ.

1.4. Tổng kết chương

Chương I đã trình bày bức tranh tổng quan về đề tài "**Phát triển hệ thống chat ngang hàng P2P**". Thông qua việc phân tích những hạn chế cố hữu của kiến trúc Client - Server truyền thống như nguy cơ thất cổ chai và điểm lỗi duy nhất, chương này đã làm rõ tính cấp thiết của việc áp dụng mô hình Hybrid P2P như một giải pháp thay thế tối ưu.

Đồng thời, chương đã xác định rõ ràng các mục tiêu nghiên cứu trọng tâm, bao gồm việc xây dựng các cơ chế giao tiếp nền tảng như phát hiện mạng, định tuyến tin nhắn cho đến các tính năng nâng cao: truyền tệp tin, mã hóa đầu cuối, cơ chế lưu trữ và chuyển tiếp Store-and-Forward. Bên cạnh đó, các giới hạn cụ thể về mặt công nghệ, môi trường triển khai và phạm vi chức năng cũng được quy định chặt chẽ nhằm đảm bảo tính khả thi của đề án trong khuôn khổ môn học.

Những nền tảng tổng quan này sẽ đóng vai trò định hướng xuyên suốt, làm cơ sở vững chắc để tiếp tục đi sâu vào phân tích lý thuyết ở Chương II và tiến hành thiết kế, xây dựng hệ thống chi tiết ở Chương III.

CHƯƠNG II: CƠ SỞ LÝ THUYẾT

2.1. Mô hình mạng ngang hàng Peer-to-Peer

2.1.1. Khái niệm và đặc trưng cơ bản

Mô hình mạng ngang hàng Peer-to-Peer, viết tắt là P2P, là một kiến trúc hệ thống ứng dụng phân tán, trong đó các khối lượng tính toán hoặc nhiệm vụ lưu trữ được phân phối đều cho các thành viên tham gia mạng. Các thành viên này được định nghĩa là các nút ngang hàng.

Khác biệt hoàn toàn với mô hình Client-Server truyền thống - nơi tồn tại sự phân định rạch ròi về mặt chức năng giữa thiết bị yêu cầu tài nguyên (máy khách) và thiết bị cung cấp dịch vụ (máy chủ). Trong mạng P2P, mỗi nút mạng đều được cấp quyền bình đẳng và đảm nhận tư cách kép. Cụ thể, mỗi thực thể vừa chia sẻ tài nguyên phần cứng của bản thân như băng thông, không gian bộ nhớ, chu kỳ CPU trực tiếp cho các thực thể khác dưới tư cách là một máy chủ, vừa có khả năng yêu cầu tài nguyên từ các nút khác dưới tư cách là một máy khách.

Những đặc trưng kỹ thuật nổi bật của kiến trúc P2P bao gồm:

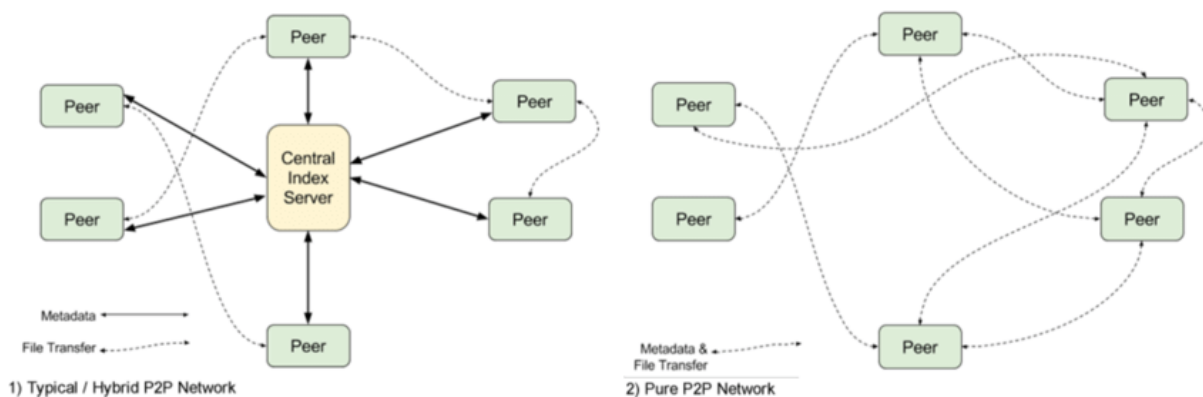
- Tính phân tán: Hệ thống không tập trung quyền kiểm soát và lưu trữ vào một điểm vật lý duy nhất. Đặc tính này giúp hệ thống loại trừ rủi ro xảy ra "nút thắt cổ chai" (bottleneck) về băng thông và giảm thiểu độ trễ truy xuất khi lưu lượng trao đổi mạng gia tăng đột biến.
- Khả năng tự tổ chức và định tuyến: Các nút mạng sở hữu khả năng tự động thiết lập liên kết, duy trì cấu trúc tô-pô và tự động định tuyến thông điệp mà không cần sự can thiệp từ quản trị viên hệ thống.
- Khả năng mở rộng tuyến tính: Trái ngược với mô hình tập trung - hiệu năng giảm dần khi số lượng Client tăng lên. Trong mạng P2P, khi số lượng nút tham gia càng nhiều, tổng tài nguyên của toàn mạng lưới càng lớn, dẫn đến năng lực xử lý tổng thể của hệ thống tăng lên theo tỷ lệ thuận.

- Tính biến động trạng thái: Đây là một thách thức đặc thù của mạng phân tán, xảy ra khi các nút liên tục gia nhập và rời khỏi mạng lưới một cách ngẫu nhiên và không dự báo trước. Hệ thống P2P yêu cầu phải có cơ chế quản lý trạng thái hiệu quả để duy trì tính toàn vẹn của đồ thị kết nối.

2.1.2. Phân loại các kiến trúc mạng P2P

Dựa trên mức độ tập trung của cơ sở dữ liệu định tuyến và phương thức truy vấn thông tin, mạng P2P được phân thành hai mô hình kiến trúc chủ đạo:

- Mạng P2P thuần túy (Pure P2P): Trong kiến trúc này, hoàn toàn không tồn tại bất kỳ máy chủ trung tâm hay thực thể phân phối chỉ mục nào. Tất cả các nút hoàn toàn bình đẳng và kết nối dựa trên cơ chế láng giềng ngẫu nhiên.
 - Ưu điểm: Loại bỏ triệt để điểm lỗi rủi ro duy nhất SPOF, cung cấp mức độ ẩn danh và độ bền vững tối đa cho hệ thống.
 - Nhược điểm: Quá trình "khám phá mạng" (Peer Discovery) và tìm kiếm tài nguyên có độ phức tạp cao. Thuật toán tìm kiếm thường dựa trên cơ chế truyền bá ngập lụt, gây lãng phí băng thông nghiêm trọng, dễ dẫn đến hiện tượng "bão thông điệp" và thời gian hội tụ dữ liệu chậm.
- Mạng P2P lai (Hybrid P2P): Đây là mô hình kiến trúc được thiết kế nhằm giao thoa và tận dụng ưu điểm của cả mô hình Client-Server và Pure P2P. Hệ thống triển khai một máy chủ trung tâm thường được định danh là Tracker Server, Index Server hoặc Bootstrap Server giữ vai trò quản lý không gian tên và điều phối định tuyến. Máy chủ này chỉ lưu trữ thông tin meta như địa chỉ IP, Port, trạng thái khả dụng của các nút mà không tham gia vào quá trình vận chuyển dữ liệu thực tế.
 - Ưu điểm: Quá trình truy vấn và khám phá mạng đạt hiệu suất tối ưu với độ trễ thấp với độ phức tạp thời gian thường đạt $O(1)$. Tiết kiệm tối đa tài nguyên băng thông do không phát sinh các gói tin Flooding.
 - Cơ chế hoạt động: Nút mạng thực hiện thủ tục đăng ký thông tin với Tracker. Sau khi thu thập được địa chỉ IP/Port của đích đến, các luồng dữ liệu chính như tệp tin, thông điệp sẽ được truyền tải trực tiếp giữa các nút ngang hàng.



Hình 2.1: Sơ đồ so sánh hai mô hình kiến trúc P2P

2.1.3. Cơ sở lý thuyết áp dụng vào đề án

Trong khuôn khổ đề án, kiến trúc Hybrid P2P được lựa chọn làm nền tảng phát triển. Sự lựa chọn này xuất phát từ yêu cầu cân đối giữa hiệu năng định tuyến và tính chất luân chuyển dữ liệu nặng của một hệ thống giao tiếp theo thời gian thực. Cấu trúc này được ánh xạ trực tiếp vào mã nguồn của dự án như sau:

- Thực thể Tracker Server: Đảm nhiệm vai trò giải quyết bài toán “khám phá mạng” thông qua việc quản lý các thông điệp điều khiển bao gồm lệnh REGISTER (đăng ký), LEAVE (rời mạng) và PING (nhịp tim). Tracker liên tục duy trì một bảng băm chứa trạng thái toàn cục của mạng, cho phép cập nhật theo thời gian thực.
- Thực thể Peer Node: Triển khai cơ chế giao tiếp trực tiếp đa luồng. Thông lượng dữ liệu tải trọng lớn như thông điệp văn bản mã hóa hay tệp tin nhị phân được định tuyến và vận chuyển qua các phiên kết nối TCP độc lập giữa các Peer. Thiết kế này tuân thủ nguyên lý phân tán tải trọng của mạng ngang hàng, đồng thời tận dụng Tracker Server như một cơ chế dự phòng an toàn để xử lý các gói tin khi phát sinh sự cố đứt gãy kết nối cục bộ do nút đích ngoại tuyến.

2.2. Giao thức điều khiển truyền vận TCP và Lập trình Socket

2.2.1. Giao thức TCP

TCP là một trong những giao thức cốt lõi và quan trọng nhất thuộc bộ giao thức liên mạng, hoạt động tại tầng giao vận của mô hình tham chiếu OSI. Không giống như UDP vốn ưu tiên tốc độ nhưng chấp nhận mất mát dữ liệu, TCP được thiết kế với triết lý ưu

tiên sự toàn vẹn và độ tin cậy tuyệt đối của luồng dữ liệu. Những đặc trưng kỹ thuật ưu việt của giao thức TCP bao gồm:

- Giao tiếp hướng kết nối và bắt tay 3 bước: Trước khi bất kỳ byte dữ liệu nào được truyền tải, TCP yêu cầu thiết lập một kênh kết nối logic và đồng bộ hóa trạng thái giữa điểm gửi và điểm nhận thông qua thủ tục bắt tay 3 bước: SYN -> SYN-ACK -> ACK. Cơ chế này đảm bảo rằng nút đích đang trong trạng thái sẵn sàng tiếp nhận luồng dữ liệu, loại bỏ tình trạng gửi thông điệp vào "hư không" khi nút đích đã ngoại tuyến.
- Kiểm soát luồng và tắc nghẽn: TCP triển khai thuật toán Sliding Window để điều tiết tốc độ phát dữ liệu. Nếu nút đích xử lý chậm hoặc băng thông mạng bị nghẽn, TCP sẽ tự động giảm tốc độ gửi, ngăn ngừa hiện tượng tràn bộ đệm vốn rất dễ xảy ra khi truyền tải tệp tin dung lượng lớn.
- Đảm bảo tính toàn vẹn và tái tổ hợp: TCP duy trì cơ chế báo nhận và kiểm tra tổng cho từng phân đoạn dữ liệu. Nếu một gói tin bị suy hao do nhiễu vật lý hoặc thất lạc trên đường truyền, giao thức sẽ tự động thực hiện tiến trình truyền lại. Hơn nữa, thông qua việc đánh số thứ tự, hệ thống đích có thể tái tổ hợp các phân mảnh dữ liệu về đúng trình tự ban đầu.
- Truyền dữ liệu dạng luồng và hiện tượng dính gói tin: Một đặc điểm hệ trọng của TCP là nó nhìn nhận dữ liệu như một dòng chảy byte liên tục chứ không phải là các thông điệp rời rạc độc lập. Do các thuật toán tối ưu hóa mạng như thuật toán Nagle, nhiều thông điệp gửi đi liên tiếp có thể bị gộp chung vào một gói tin TCP duy nhất hoặc một thông điệp lớn có thể bị cắt làm nhiều mảnh. Điều này đòi hỏi tầng ứng dụng phải tự thiết kế cơ chế phân định ranh giới gói tin.

2.2.2. Giao diện lập trình Socket

Socket là một giao diện lập trình ứng dụng cấp thấp do hệ điều hành cung cấp, tạo ra một thiết bị cuối tiêu chuẩn để các tiến trình trên các thiết bị vật lý phân tán có thể trao đổi luồng dữ liệu hai chiều qua mạng máy tính. Một liên kết định tuyến Socket TCP được định danh duy nhất bởi sự kết hợp của hai tham số: Địa chỉ định tuyến IP xác định

thiết bị trên mạng và số hiệu Cổng - Port xác định tiến trình dịch vụ cụ thể đang chạy trên thiết bị đó.

Vòng đời và mô hình hoạt động của Socket TCP trong kiến trúc mạng thường được chia thành hai vai trò tương hỗ:

- Chu trình Server: Khởi tạo *socket()* -> Gắn kết *bind()* vào một IP và Port cố định -> Chuyển sang trạng thái chờ *listen()* -> Chấp nhận kết nối *accept()* - hàm này có tính chất chặn cho đến khi có một Client yêu cầu kết nối -> Giao tiếp *recv()/send()*.
- Chu trình Client: Khởi tạo *socket()* -> Gửi yêu cầu thiết lập kết nối *connect()* tới IP/Port của Server đích -> Giao tiếp *send()/recv()*.

Cơ sở ứng dụng: Nhằm hiện thực hóa bản chất "vừa là máy khách, vừa là máy chủ" của mô hình P2P, cũng như đảm bảo tính trọn vẹn của luồng dữ liệu, hệ thống áp dụng linh hoạt các khái niệm trên:

1. Xử lý ranh giới gói tin TCP: Mặc dù sử dụng chuẩn cấu trúc dữ liệu JSON, hệ thống quy định nghiêm ngặt việc nối thêm ký tự ngắt dòng “\n” vào cuối mỗi thông điệp trước khi gọi hàm *sendall()*. Tại điểm nhận, thuật toán sẽ liên tục đọc luồng byte TCP vào bộ đệm và tiến hành tách chuỗi dựa trên ký tự “\n”, giải quyết triệt để hiện tượng dính/phân mảnh gói tin TCP ở tầng giao vận.
2. Kiến trúc đa Socket tại Peer Node: Mỗi nút mạng duy trì đồng thời một Socket lắng nghe chạy trên một luồng ngầm để tiếp nhận thông điệp từ mạng, và khởi tạo liên tục các Socket khách dạng ngắn hạn mỗi khi người dùng có nhu cầu gửi tin nhắn hoặc truyền file.
3. Cơ chế Timeout và bắt ngoại lệ: Trong môi trường mạng phân tán, hiện tượng chập chờn diễn ra thường xuyên. Đồ án áp dụng hàm *settimeout()* cho các Socket để phá vỡ tính chất blocking mặc định. Nếu nút đích tắt máy đột ngột, TCP Handshake sẽ thất bại và sinh ra ngoại lệ *ConnectionRefusedError*; nếu mạng quá tải khiến quá trình báo nhận bị trễ, ngoại lệ *socket.timeout* sẽ được kích hoạt. Các ngoại lệ này được hệ thống xử lý để tự động kích hoạt tính năng gửi tin nhắn ngoại tuyến lên Tracker Server thay vì làm sập ứng dụng nút.

2.3. Lập trình đa luồng và cơ chế đồng bộ hóa

2.3.1. Kỹ thuật đa luồng và xử lý đồng thời

Trong kiến trúc hệ điều hành hiện đại, tiến trình được định nghĩa là một thực thể chương trình đang trong trạng thái thực thi, được hệ điều hành cấp phát một không gian địa chỉ bộ nhớ độc lập. Luồng là một tiến trình con, đóng vai trò là đơn vị lập lịch nhỏ nhất của CPU. Một tiến trình có thể chứa nhiều luồng thực thi song song, các luồng này chia sẻ chung không gian tài nguyên, tệp tin mô tả và bộ nhớ cấp phát của tiến trình mẹ. Việc chuyển ngữ cảnh giữa các luồng có chi phí tính toán thấp hơn rất nhiều so với chuyển ngữ cảnh giữa các tiến trình.

Trong lập trình mạng cơ bản, các API tương tác với Socket để đọc dữ liệu thường mang đặc tính chặn I/O. Khi một luồng gọi các phương thức này, nó sẽ chuyển từ trạng thái sẵn sàng sang trạng thái chờ cho đến khi sự kiện mạng thực sự xảy ra. Nếu ứng dụng chỉ chạy trên một luồng duy nhất, toàn bộ vòng đời chương trình sẽ bị đình trệ tại lệnh chặn, dẫn đến hiện tượng "đóng băng" giao diện người dùng và không thể đáp ứng các thao tác đầu vào.

Kỹ thuật đa luồng giải quyết bài toán này thông qua phương pháp xử lý đồng thời. Bằng cách khởi tạo các luồng chuyên biệt để đảm nhiệm các tác vụ I/O-bound hay các tác vụ nặng về thời gian chờ thiết bị ngoại vi/mạng, luồng chính có thể liên tục duy trì vòng lặp sự kiện, đảm bảo ứng dụng luôn trong trạng thái phản hồi theo thời gian thực.

2.3.2. Vấn đề tranh chấp tài nguyên và đồng bộ hóa

Sự chia sẻ không gian bộ nhớ mang lại hiệu suất cao cho đa luồng nhưng đồng thời phát sinh thách thức kỹ thuật lớn nhất: Tranh chấp tài nguyên. Hiện tượng này xảy ra khi bộ lập lịch của hệ điều hành tự do hoán đổi ngữ cảnh giữa các luồng tại những thời điểm không lường trước. Nếu nhiều luồng đồng thời cố gắng đọc và ghi vào một cấu trúc dữ liệu dùng chung, ví dụ: mảng danh bạ, biến đếm,... mà không có cơ chế rào chắn, dữ liệu sẽ bị ghi đè sai lệch, dẫn đến tính trạng bất đồng bộ và làm sập ứng dụng.

Để giải quyết tình trạng này, khoa học máy tính đưa ra khái niệm vùng găng (Critical Section) - là đoạn mã nơi các tài nguyên chia sẻ được truy cập. Các tiến trình hoặc luồng khi đi vào vùng găng bắt buộc phải áp dụng các cơ chế đồng bộ hóa, tiêu biểu nhất là

khóa loại trừ tương hỗ. Khi một luồng giữ khóa, tất cả các luồng khác muốn vào vùng vắng sẽ bị hệ điều hành xếp vào hàng đợi chờ cho đến khi khóa được giải phóng.

2.3.3. Mô hình Producer-Consumer Problem và hàng đợi an toàn luồng

Trong thiết kế hệ thống phân tán và giao diện đồ họa, các luồng mạng không được phép can thiệp trực tiếp vào dữ liệu kết xuất của luồng giao diện để tránh vi phạm cấu trúc bộ nhớ của hệ thống đồ họa. Mỗi quan hệ giữa luồng mạng và luồng giao diện được mô hình hóa thành một bài toán kinh điển trong Khoa học máy tính: Bài toán Người sản xuất - Người tiêu thụ.

- Người sản xuất (Producer): Đại diện cho các luồng mạng. Luồng này liên tục lắng nghe và sinh ra các gói tin dữ liệu mới.
- Người tiêu thụ (Consumer): Đại diện cho luồng giao diện chính. Luồng này lấy dữ liệu đã nhận để in ấn hoặc kết xuất ra màn hình.
- Bộ đệm trung gian (Bounded Buffer): Hệ thống triển khai một cấu trúc hàng đợi tuân thủ nguyên tắc “First-In, First-Out” làm bộ đệm giao tiếp.

Để mô hình này hoạt động bền vững, hàng đợi phải là một hàng đợi an toàn luồng. Điều này có nghĩa là cấu trúc dữ liệu hàng đợi đã được tích hợp sẵn các nguyên thủy đồng bộ hóa như Mutex và Condition Variables ở tầng hệ thống. Khi Producer muốn thêm dữ liệu hoặc Consumer muốn lấy dữ liệu, hàng đợi sẽ tự động khóa vùng nhớ cục bộ, đảm bảo tính toàn vẹn tuyệt đối của dữ liệu mà không yêu cầu lập trình viên phải tự quản lý các cơ chế Lock phức tạp.

Cơ sở ứng dụng trong đồ án: Kiến trúc của đồ án tuân thủ nghiêm ngặt mô hình Producer - Consumer kết hợp đa luồng. Cụ thể:

1. Luồng lắng nghe mạng hoạt động độc lập ở chế độ ngầm, bảo vệ ứng dụng khỏi hiện tượng chặn I/O.
2. Việc chuyển giao thông điệp từ lớp mạng lên lớp giao diện được trừu tượng hóa thông qua một phiên bản hàng đợi an toàn luồng. Mọi thao tác tương tác giữa các thành phần kiến trúc đều tuân thủ nguyên lý truyền thông điệp thay vì chia sẻ bộ nhớ trực tiếp, loại trừ triệt để nguy cơ xuất hiện lỗi Race Condition trong suốt vòng đời vận hành của ứng dụng.

2.4. Mật mã học và giao thức bảo mật đầu cuối

2.4.1. Vai trò của mật mã học trong hệ thống mạng mở

Mật mã học là nền tảng cốt lõi đảm bảo ba trụ cột của an toàn thông tin: Tính bảo mật, tính toàn vẹn và tính sẵn sàng. Trong môi trường mạng Internet - một không gian mở và không đáng tin cậy, các luồng dữ liệu truyền tải dưới dạng bản rõ tiềm ẩn rủi ro cực kỳ lớn trước các cuộc tấn công như: Nghe lén gói tin tại các điểm định tuyến công cộng hoặc tấn công trung gian nhằm đánh cắp hoặc thay đổi nội dung thông điệp. Việc áp dụng các giao thức mật mã học là yêu cầu bắt buộc để cô lập dữ liệu khỏi sự dòm ngó của các thực thể không được cấp phép.

2.4.2. Nguyên lý hoạt động của mã hóa đối xứng

Mã hóa đối xứng là phương pháp mật mã hóa mà trong đó tiến trình biến đổi dữ liệu bản rõ thành bản mã và tiến trình hoàn nguyên bản mã về bản rõ đều sử dụng chung một khóa bí mật duy nhất.

Theo mô hình toán học, nếu P là bản rõ, K là khóa bí mật và C là bản mã, hệ thống mã hóa đối xứng được mô tả bởi hai hàm:

- Hàm mã hóa (Encryption): $C = E_K(P)$
- Hàm giải mã (Decryption): $P = D_K(C)$

Đặc điểm kỹ thuật của mã hóa đối xứng:

- Ưu điểm: Các thuật toán đối xứng thường sử dụng các phép toán bit logic kết hợp với bảng thế, do đó hiệu năng tính toán cực kỳ cao, tốn ít tài nguyên CPU và độ trễ thấp. Điều này làm cho mã hóa đối xứng trở thành giải pháp tối ưu số một để mã hóa các luồng dữ liệu thời gian thực có kích thước lớn như luồng video stream, truyền tệp tin, chat liên tục.
- Nhược điểm: Khó khăn lớn nhất nằm ở khâu trao đổi khóa. Để hai nút có thể giao tiếp, chúng phải tìm cách chia sẻ khóa K một cách an toàn trên một kênh truyền có thể đang bị theo dõi. Ngoài ra, trong một mạng có N người dùng giao tiếp chéo lẫn nhau, hệ thống yêu cầu tới $\frac{N(N-1)}{2}$ khóa khác nhau, gây khó khăn trong việc quản trị quy mô lớn.

2.4.3. Tiêu chuẩn AES và khung bảo mật Fernet

Trong số các thuật toán mã hóa đối xứng, AES (Advanced Encryption Standard) hiện đang là tiêu chuẩn bảo mật toàn cầu được Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) chứng nhận. AES hoạt động theo cơ chế mã hóa khối, xử lý dữ liệu theo từng khối cố định.

Thư viện cryptography của Python cung cấp một mô-đun cấp cao mang tên Fernet. Fernet không chỉ đơn thuần là một thuật toán mã hóa, mà là một "công thức" mật mã học hoàn chỉnh kết hợp nhiều nguyên thủy mật mã nhằm đảm bảo cả tính bảo mật lẫn tính toàn vẹn của dữ liệu:

1. Mã hóa AES-128 ở chế độ CBC (Cipher Block Chaining): Dữ liệu được mã hóa bằng thuật toán AES. Chế độ CBC sử dụng một Vector khởi tạo ngẫu nhiên để đảm bảo rằng cùng một bản rõ nếu mã hóa nhiều lần sẽ tạo ra các bản mã hoàn toàn khác nhau, chống lại các phương pháp phân tích tần suất.
2. Xác thực thông điệp HMAC (Hash-based Message Authentication Code) bằng SHA-256: Tính bảo mật là chưa đủ nếu kẻ tấn công cố tình sửa đổi các bit trong bản mã để làm sai lệch dữ liệu giải mã. Fernet giải quyết triệt để vấn đề này bằng cách sinh ra một chữ ký điện tử đính kèm vào cuối bản mã. Tại nút đích, hệ thống sẽ kiểm tra chữ ký này trước khi tiến hành giải mã. Nếu bản mã bị can thiệp dù chỉ 1 bit, thuật toán sẽ từ chối giải mã và ném ra ngoại lệ.

2.4.4. Mô hình bảo mật đầu cuối trong hệ thống

Bảo mật đầu cuối (E2EE) là phương pháp luận bảo mật trong đó chỉ có người dùng ở hai đầu của quá trình giao tiếp mới sở hữu khóa để mã hóa và giải mã thông điệp. Không một thực thể trung gian nào - kể cả nhà cung cấp dịch vụ Internet, hay bản thân máy chủ điều phối mạng - có thể can thiệp hay đọc được nội dung gốc.

Cơ sở ứng dụng trong đồ án: Hệ thống đồ án P2P Chat đã tích hợp mô hình E2EE thông qua cơ chế mật mã Fernet như sau:

- Giai đoạn mã hóa: Khi người dùng khởi tạo tin nhắn, chuỗi ký tự sẽ được mã hóa lập tức bằng hàm `cipher_suite.encrypt()`. Sau đó, bản mã mới được đóng gói vào đối tượng JSON để đẩy ra đường truyền TCP.

- Vai trò trung gian mù: Trong trường hợp nút đích ngoại tuyến, cơ chế Store-and-Forward sẽ chuyển hướng gói tin lên Tracker Server. Lúc này, Tracker Server đóng vai trò như một kho lưu trữ "mù". Nó chỉ lưu giữ bản mã ciphertext mà không sở hữu khóa bí mật để giải mã nội dung, đảm bảo tuyệt đối quyền riêng tư của người dùng.
- Giới hạn triển khai: Thay vì xây dựng cơ sở hạ tầng khóa công khai và thuật toán trao đổi khóa động qua thuật toán bất đối xứng RSA/Diffie-Hellman, đồ án tập trung vào kiến trúc phân tán nên đã đơn giản hóa bài toán bằng cách sử dụng một khóa bí mật được chia sẻ trước. Sự đánh đổi này giúp giảm thiểu độ phức tạp giao thức nhưng vẫn đảm bảo chứng minh được tính đúng đắn của quy trình mã hóa và giải mã hai chiều trong hệ thống P2P.

2.5. Tuần tự hóa dữ liệu và mã hóa Base64

2.5.1. Khái niệm và vai trò của tuần tự hóa dữ liệu

Trong môi trường điện toán, dữ liệu được thao tác và lưu trữ trên bộ nhớ chính dưới dạng các cấu trúc dữ liệu phức tạp mang tính phụ thuộc cao vào ngôn ngữ lập trình cụ thể như Object, Dictionary, hay List. Tuy nhiên, môi trường mạng máy tính hay các luồng I/O lại chỉ chấp nhận vận chuyển luồng byte tuyến tính một chiều. Quá trình biến đổi cấu trúc dữ liệu trên RAM thành một chuỗi byte theo tiêu chuẩn định dạng nhất định để có thể lưu trữ hoặc truyền tải qua mạng được gọi là tuần tự hóa. Quá trình ngược lại để khôi phục cấu trúc dữ liệu nguyên bản tại máy nhận được gọi là giải tuần tự hóa.

Hệ thống đồ án lựa chọn chuẩn JSON làm định dạng tuần tự hóa dữ liệu độc quyền. So với các chuẩn cũ như XML, JSON có ưu điểm vượt trội về độ gọn nhẹ, giảm overhead truyền tải, tính tương thích chéo nền tảng tuyệt đối và khả năng phân tích cú pháp cực kỳ nhanh chóng. Việc thiết kế gói tin giao tiếp dựa trên các cặp "Khóa - Giá trị" của JSON mang lại tính mở rộng linh hoạt, cho phép hệ thống bổ sung các trường dữ liệu mới mà không làm phá vỡ logic giao thức hiện tại.

2.5.2. Chuyển đổi dữ liệu nhị phân với thuật toán Base64

Hạn chế nội tại lớn nhất của chuẩn JSON là nó được thiết kế độc quyền cho việc lưu trữ văn bản, chỉ chấp nhận các ký tự tuân theo bảng mã UTF-8 hoặc ASCII. JSON hoàn

toàn không có khả năng chứa trực tiếp các chuỗi byte nhị phân thô - vốn là bản chất cấu thành nên tệp tin hình ảnh, tệp tin nén hay tài liệu PDF.

Để giải quyết mâu thuẫn này, thuật toán Base64 được áp dụng như một kỹ thuật chuyển đổi trung gian. Thuật toán hoạt động dựa trên cơ sở toán học cắt dán bit:

1. Base64 sẽ đọc luồng dữ liệu nhị phân nguyên bản theo từng khối 3 byte, tương đương 24 bit.
2. Chuỗi 24 bit này sau đó được chia lại thành 4 khối, mỗi khối mang kích thước 6-bit.
3. Mỗi khối 6-bit, mang giá trị từ 0 đến 63, sẽ được ánh xạ vào một tập hợp gồm 64 ký tự an toàn tiêu chuẩn quốc tế bao gồm các chữ cái in hoa A-Z, in thường a-z, chữ số 0-9, cùng với dấu + và /.
4. Nếu chuỗi byte cuối cùng không chia hết cho 3, thuật toán sẽ tự động bổ sung ký tự đệm để đảm bảo tính toàn vẹn.

Sự đánh đổi của phương pháp này là việc chia 3 byte thành 4 byte sẽ khiến kích thước dữ liệu vận chuyển bị phình to khoảng 33% so với bản gốc nhị phân, đồng thời tiêu tốn thêm tài nguyên tính toán của CPU cho quá trình mã hóa.

Cơ sở ứng dụng trong đồ án: Nhằm duy trì tính đồng nhất của kiến trúc mạng, hệ thống không mở thêm các Socket truyền nhị phân riêng biệt mà chuẩn hóa mọi luồng giao tiếp qua JSON. Đối với tính năng truyền tệp tin, ứng dụng nút gửi sẽ thực hiện thao tác nạp tệp tin vào bộ nhớ, gọi hàm *base64.b64encode()* để chuyển chuỗi byte sang văn bản, rồi nhúng toàn bộ vào trường payload của JSON. Tại ứng dụng nút đích, hàm *base64.b64decode()* được thực thi trên chuỗi văn bản nhận được để hoàn nguyên chính xác tệp tin nhị phân và lưu xuống ổ cứng.

2.6. Lập trình giao diện hướng sự kiện với Tkinter

2.6.1. Mô hình lập trình hướng sự kiện

Tkinter là thư viện cung cấp bộ công cụ giao diện người dùng đồ họa tiêu chuẩn, được tích hợp sâu vào hệ sinh thái ngôn ngữ Python. Điểm khác biệt lớn nhất giữa việc phát

triển phần mềm giao diện và phần mềm chạy trên dòng lệnh là sự chuyển dịch từ mô hình lập trình tuần tự sang mô hình lập trình hướng sự kiện.

Thay vì thực thi mã lệnh từ trên xuống dưới rồi kết thúc, một ứng dụng đồ họa hướng sự kiện khởi chạy và duy trì một chu trình lặp vô hạn mang tên vòng lặp sự kiện. Trong Tkinter, phương thức này được kích hoạt thông qua lời gọi hàm *mainloop()*. Vòng lặp này giữ nhiệm vụ giám sát hệ điều hành để phát hiện mọi tác động ngoại vi, ví dụ như thao tác nhấp chuột, gõ bàn phím, hay các tín hiệu làm mới cửa sổ. Khi một sự kiện được kích hoạt, Event Loop sẽ định tuyến luồng thực thi đến một hàm xử lý hồi quy tương ứng đã được lập trình viên đăng ký từ trước. Sau khi xử lý xong, hệ thống ngay lập tức quay lại trạng thái chờ sự kiện tiếp theo.

2.6.2. Bài toán an toàn luồng trong GUI và Cơ chế hồi vòng

Theo thiết kế hệ thống gốc của thư viện Tcl/Tk, lớp nền tảng của Tkinter, hệ thống không hỗ trợ cơ chế an toàn luồng. Giao thức này quy định nghiêm ngặt: Mọi thao tác làm thay đổi trạng thái của các thành phần giao diện như cập nhật đoạn văn bản tin nhắn, hiển thị hộp thoại, đổi màu nút bấm bắt buộc phải được thực thi độc quyền bởi luồng chính, nơi nắm giữ đặc quyền sở hữu vòng lặp sự kiện. Bất kỳ nỗ lực nào từ các luồng bên ngoài nhằm trực tiếp thay đổi thuộc tính của đối tượng đồ họa sẽ dẫn đến các lỗi tranh chấp vùng nhớ, thường biểu hiện bằng lỗi treo ứng dụng hoặc sập hệ thống đột ngột.

Cơ sở ứng dụng trong đồ án: Đồ án đã thiết kế một kiến trúc cầu nối tinh tế nhằm dung hòa giữa tốc độ xử lý I/O mạng của các luồng phụ và yêu cầu Thread-safety của luồng chính:

1. Cô lập xử lý mạng: Các tiến trình chờ kết nối Socket được tách biệt hoàn toàn vào các luồng nền ngầm. Do đó, ứng dụng không bao giờ bị đình trệ khi luồng mạng đang chờ đợi phản hồi.
2. Cầu nối hàng: Khi luồng mạng nhận được tin nhắn, nó không gọi hàm cập nhật màn hình mà đóng gói tin nhắn đẩy vào queue.Queue.
3. Cơ chế Polling: Để luồng chính biết được khi nào có tin nhắn mới trong Queue, đồ án sử dụng phương thức `.after(ms, callback)` của Tkinter. Phương thức này ra

lệnh cho Event Loop cứ mỗi khoảng thời gian cố định sẽ tự động kích hoạt hàm cập nhật giao diện. Luồng chính lúc này đóng vai trò là Consumer, lấy toàn bộ thông điệp đang có trong Queue kết xuất lên màn hình một cách an toàn và hợp lệ về mặt luồng thực thi. Giải pháp này mang lại một trải nghiệm ứng dụng mạng P2P mượt mà theo thời gian thực.

2.7. Tổng kết chương

Chương II đã hoàn thiện việc trình bày có hệ thống một khung lý thuyết vững chắc, kết hợp liên ngành giữa khoa học mạng máy tính, kiến trúc hệ điều hành và nguyên lý an toàn thông tin, nhằm phục vụ cho mục tiêu phát triển hệ thống mạng ngang hàng.

Nền tảng của hệ thống được xây dựng trên mô hình kiến trúc Hybrid P2P, mang lại ưu thế tuyệt đối về tốc độ truy vấn trạng thái so với Pure P2P, kết hợp với giao thức TCP ở tầng giao vận để bảo chứng cho tính toàn vẹn của đường truyền dữ liệu không suy hao. Sự chuyển dịch dữ liệu giữa các thực thể vật lý được giải quyết linh hoạt nhờ chuẩn tuần tự hóa JSON và thuật toán chuyển đổi nhị phân Base64. Ở tầng vi mô bên trong mỗi nút mạng, việc ứng dụng thiết kế đa luồng và mô hình Producer-Consumer Queue kết hợp với kỹ thuật giao diện hướng sự kiện đã triệt tiêu hoàn toàn các nhược điểm thất cổ chai I/O và xung đột vùng nhớ. Cuối cùng, việc triển khai tiêu chuẩn mã hóa đối xứng AES đảm bảo thông điệp đạt mức bảo mật đầu cuối, đáp ứng trọn vẹn các yêu cầu khắt khe về quyền riêng tư trên không gian mạng.

Những phân tích lý luận chuyên sâu trong chương này chính là kim chỉ nam quan trọng, luận giải cho mọi quyết định thiết kế kỹ thuật sẽ được hiện thực hóa ở mã nguồn phần mềm tại Chương III.

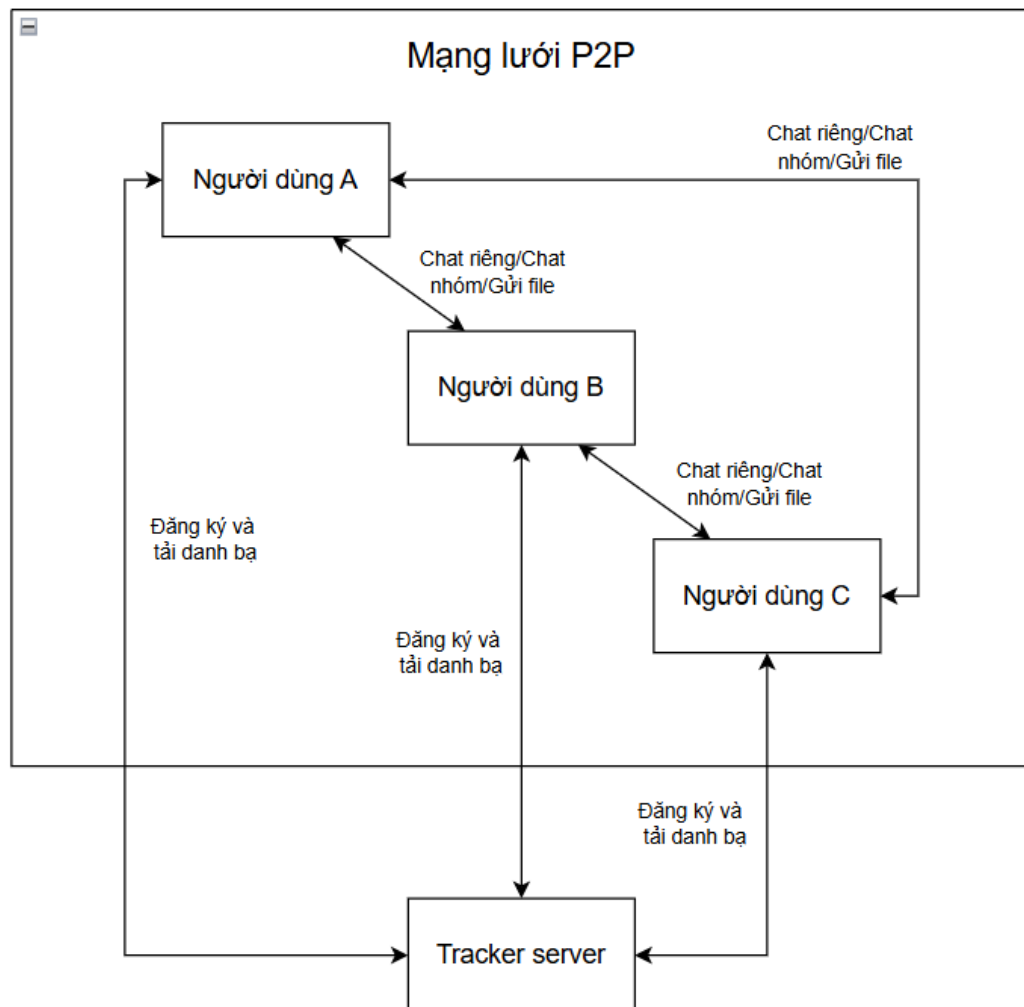
CHƯƠNG III: XÂY DỰNG HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống

3.1.1. Tổng quan kiến trúc hệ thống

Dựa trên các cơ sở lý thuyết đã được phân tích tại trước đó, chương này sẽ đi sâu vào quá trình hiện thực hóa hệ thống chat ngang hàng P2P. Chương này trình bày chi tiết về kiến trúc tổng thể, cấu trúc giao thức thông điệp tự định nghĩa, các cơ chế xử lý lỗi và giải pháp tích hợp giao diện người dùng đồ họa đa luồng.

Kiến trúc tổng thể của hệ thống phân tách rõ ràng dòng chảy dữ liệu thành hai luồng tương tác song song, được minh họa cụ thể qua sơ đồ dưới đây.



Hình 3.1: Sơ đồ kiến trúc tổng quan hệ thống

Như sơ đồ kiến trúc minh họa, hệ thống bao gồm một Tracker Server độc lập và một Mạng lưới P2P bao gồm nhiều thực thể người dùng. Các luồng tương tác được quy định như sau:

- Luồng giao tiếp với Tracker Server: Các nút mạng khi khởi động sẽ thiết lập kết nối hướng tâm đến Tracker Server. Luồng giao tiếp này phục vụ mục đích khai báo gia nhập mạng và đồng bộ hóa trạng thái mạng. Các thao tác này là các thông điệp điều khiển yêu cầu lượng băng thông rất nhỏ. Điều này giúp hệ thống đạt được trạng thái hội tụ định tuyến cực kỳ nhanh chóng thay vì phải dùng thuật toán Flooding kém hiệu quả.
- Luồng giao tiếp nội bộ mạng lưới P2P: Sau khi đã tải danh bạ và biết được địa chỉ IP/Port của nhau, các Người dùng thiết lập kết nối chéo trực tiếp với nhau tạo thành một mạng lưới P2P khép kín. Mọi tác vụ mang tải trọng dữ liệu lớn bao gồm nhắn tin trực tiếp, nhắn tin đa luồng và trao đổi tệp tin nhị phân đều được vận chuyển hoàn toàn trực tiếp giữa các nút mạng ngang hàng. Tracker Server không bị ép phải chuyển tiếp các dữ liệu này, qua đó triệt tiêu hoàn toàn rủi ro thất cổ chai về băng thông mạng.

3.1.2. Máy chủ dẫn đường (Tracker Server)

Tracker Server đóng vai trò là một điểm neo tĩnh trong hệ thống, thường được cấu hình để vận hành liên tục tại một địa chỉ IP và cổng cố định, công khai để các nút mạng có thể tham chiếu đến ngay khi vừa khởi động.

Được thiết kế theo nguyên lý tối giản, máy chủ dẫn đường không tham gia vào quá trình mã hóa, giải mã hay vận chuyển tin nhắn trực tiếp giữa các peer khi mạng hoạt động bình thường. Nhiệm vụ cốt lõi của thành phần này bao gồm:

- Hệ thống quản lý danh bạ trực tuyến: Liên tục duy trì một cấu trúc dữ liệu bảng băm nhằm ánh xạ định danh của mỗi nút với thông tin định tuyến vật lý thực tế của nút đó bao gồm địa chỉ ip, port và dấu thời gian last_ping.
- Cơ chế giám sát trạng thái: Quản lý vòng đời của các nút thông qua một luồng ngầm. Nó liên tục đối chiếu dấu thời gian của các gói tin nhip tim gửi về, từ đó

phát hiện, vô hiệu hóa và dọn dẹp các nút ngắt kết nối đột ngột nhằm duy trì tính chính xác tuyệt đối của bảng định tuyến toàn cục.

- **Hộp thư trung chuyển ngoại tuyến:** Đóng vai trò là một cơ chế dự phòng an toàn. Khi các kết nối P2P thất bại do nút đích ở trạng thái ngoại tuyến, Tracker Server cung cấp một không gian lưu trữ tạm thời để chứa các thông điệp mã hóa. Nó sẽ tự động giải phóng và chuyển tiếp dữ liệu về nút đích ngay khi nút này khai báo trực tuyến trở lại.

3.1.3. Nút mạng ngang hàng (Peer Node)

Thành phần nút mạng là phần mềm đầu cuối được cài đặt và thực thi trên các thiết bị của người dùng. Trong mô hình Hybrid P2P, mỗi nút mạng là một thực thể máy tính độc lập, mang đặc tính kép và có cấu trúc phức tạp hơn rất nhiều so với một ứng dụng Client thông thường:

- **Tư cách nút yêu cầu:** Khi khởi chạy, tiến trình đóng vai trò là một Client tạo các kết nối TCP ngắn hạn tới Tracker Server để đăng ký thông tin và tải danh bạ định tuyến. Đồng thời, mỗi khi người dùng có nhu cầu gửi tin nhắn hoặc truyền tệp tin, nút mạng sẽ tự động mở các Socket khách động để chủ động kết nối tới IP và Port của nút đích mong muốn.
- **Tư cách nút phục vụ:** Ngay khi khởi động thành công, tiến trình sẽ cấp phát và gán một Socket vào một cổng hệ thống cụ thể, duy trì một luồng ngầm liên tục lắng nghe. Tư cách này cho phép nút mạng sẵn sàng tiếp nhận hàng loạt các yêu cầu kết nối TCP đổ về từ bất kỳ nút nào khác trong mạng vào bất kỳ thời điểm nào.

Sự phân tách và thực thi song song hai tư cách này bên trong cùng một tiến trình phần mềm đòi hỏi một kiến trúc đa luồng khắt khe, đảm bảo rằng tiến trình duy trì chờ dữ liệu mạng của Socket Listener không gây ảnh hưởng đến vòng lặp sự kiện đồ họa của giao diện người dùng.

3.2. Cấu trúc giao thức trao đổi thông điệp

Giao thức truyền vận TCP bản chất là một giao thức hướng luồng byte. Nó truyền tải dữ liệu thành một dòng chảy liên tục và không tự nhận biết ranh giới của các thông điệp

rời rạc. Do đó, để hệ thống có thể hoạt động chính xác, đồ án bắt buộc phải tự định nghĩa một giao thức lớp ứng dụng nhằm tiêu chuẩn hóa cấu trúc dữ liệu và quy định cách thức các nút mạng giao tiếp với nhau.

Định dạng tuần tự hóa dữ liệu hệ thống sử dụng định dạng JSON làm chuẩn tuần tự hóa duy nhất cho mọi thông điệp trao đổi trên không gian mạng. Cấu trúc chuẩn của một gói tin được thiết kế cực kỳ chặt chẽ, bao gồm hai trường thông tin bắt buộc:

- Trường action: Là một chuỗi ký tự đóng vai trò định danh hành động mà gói tin muốn thực thi. Dựa vào chuỗi này, bộ định tuyến tại điểm nhận sẽ biết cách chuyển hướng dữ liệu đến đúng hàm xử lý.
- Trường payload: Là một đối tượng chứa các dữ liệu chi tiết phục vụ cho hành động đó. Tùy thuộc vào lệnh điều khiển được gọi, trường này sẽ mang các từ khóa dữ liệu khác nhau như tên người gửi, nội dung tin nhắn hoặc dữ liệu tệp tin.

Ví dụ về cấu trúc một gói tin nhắn văn bản thông thường được đóng gói:

```
{
  "action": "CHAT_1V1",
  "payload": {
    "sender_id": "Nghia",
    "message": "U2FsdGVkX1+vG3eR1zYw/b9M5XmKx8LpQ8ZsJ2v9K3RmWb4tN7Hq5YfG8oM4"
  }
}
```

Hình 3.2: Cấu trúc một gói tin nhắn được đóng gói

Cơ chế phân định ranh giới gói tin là một đặc trưng hệ trọng của đường truyền TCP là hiện tượng dính gói tin. Khi hai hoặc nhiều thông điệp được ứng dụng nguồn gửi đi liên tiếp với tốc độ cao, bộ đệm của hệ điều hành có xu hướng gộp chúng vào chung một luồng byte duy nhất nhằm tối ưu hóa băng thông.

Để giải quyết triệt để bài toán này và ngăn chặn lỗi phân tích cú pháp dữ liệu, hệ thống quy định bắt buộc việc nối thêm một ký tự ngắt dòng đặc biệt \n vào cuối mỗi chuỗi JSON trước khi mã hóa thành byte và đẩy ra Socket. Tại đầu nhận, bộ đệm sẽ liên tục đọc luồng byte từ đường truyền và tiến hành tách các chuỗi độc lập dựa trên vị trí của ký tự ngắt dòng. Thiết kế này là chìa khóa đảm bảo quá trình dịch ngược dữ liệu JSON diễn ra chính xác tuyệt đối.

Tập lệnh điều khiển hệ thống phân chia các gói tin thành hai nhóm tập lệnh chuyên biệt dựa trên mục tiêu giao tiếp.

Nhóm lệnh giao tiếp giữa các nút mạng và Tracker Server bao gồm:

- REGISTER: Lệnh đăng ký. Nút mạng cung cấp định danh, địa chỉ IP và cổng dịch vụ để Tracker Server ghi nhận vào bảng định tuyến tổng.
- GET_PEERS: Lệnh yêu cầu đồng bộ. Nút mạng yêu cầu Tracker Server trả về cấu trúc dữ liệu chứa toàn bộ các thành viên đang trực tuyến.
- PING: Lệnh báo cáo nhịp tim. Nút mạng định kỳ gửi tín hiệu này nhằm thông báo bản thân vẫn đang duy trì kết nối mạng ổn định.
- STORE_MSG: Lệnh lưu trữ ngoại tuyến. Nút mạng chủ động gửi thông điệp mã hóa lên Tracker Server nhờ lưu trữ tạm thời khi phát hiện người nhận mục tiêu không trực tuyến.
- OFFLINE_MSGS: Lệnh phản hồi hộp thư. Tracker Server chủ động gửi tập hợp các tin nhắn chưa đọc về cho nút mạng ngay sau khi nút này hoàn tất thủ tục đăng nhập.

Nhóm lệnh giao tiếp trực tiếp giữa các nút mạng với nhau bao gồm:

- CHAT_1V1: Lệnh gửi tin nhắn cá nhân. Gói tin mang theo định danh người gửi và nội dung đoạn văn bản đã được băm qua thuật toán mã hóa đầu cuối.
- CHAT_GROUP: Lệnh gửi tin nhắn tập thể. Tương tự lệnh cá nhân nhưng mang thêm thông tin về danh sách các thành viên trong nhóm để thiết bị nhận hiển thị đúng ngữ cảnh phòng chat.
- FILE_REQUEST, FILE_RESPONSE và FILE_DATA: Bộ ba lệnh phục vụ riêng cho quá trình truyền tệp tin nhị phân an toàn. Chúng kiểm soát luồng đồng ý hoặc từ chối nhận tệp tin từ phía người dùng đích trước khi dữ liệu thực tế được truyền đi.
- ACK: Lệnh báo nhận lớp ứng dụng. Nút đích phản hồi ngay lập tức để xác nhận thông điệp đã được xử lý thành công, giúp nút gửi biết được dữ liệu đã đến nơi an toàn và tiến hành kết thúc vòng lặp chờ.

3.3. Các cơ chế lỗi của hệ thống

3.3.1. Khám phá mạng và giám sát trạng thái hoạt động

Trong hệ thống, quy trình khám phá mạng và duy trì trạng thái được xử lý phối hợp chặt chẽ giữa các luồng nền của phần mềm đầu cuối và máy chủ dẫn đường. Để lưu trữ trạng thái người dùng theo thời gian thực với tốc độ truy xuất tức thời, đồ án đã triển khai cấu trúc từ điển dữ liệu của ngôn ngữ Python.

Ngay khi phần mềm khởi động, phương thức đăng ký sẽ sinh ra một yêu cầu kết nối tới máy chủ dẫn đường. Ở phía máy chủ, thông tin của thiết bị mới sẽ được ánh xạ vào một từ điển lưu trữ danh bạ trực tuyến. Từ điển này lưu trữ các khóa là tên định danh của người dùng, và giá trị tương ứng là một đối tượng chứa địa chỉ IP, số hiệu cổng dịch vụ cùng với dấu thời gian đăng nhập được lấy từ thư viện thời gian của hệ điều hành. Sau bước đăng ký, Nút mạng sẽ kích hoạt một luồng chạy nền độc lập làm nhiệm vụ phát lệnh đồng bộ danh bạ liên tục, tiến hành tải bản sao của Từ điển này về lưu trữ cục bộ.

Vấn đề phức tạp phát sinh khi một thiết bị mất điện hoặc đứt cáp mạng, dẫn đến việc tiến trình phần mềm bị hủy diệt mà không kịp phát đi lệnh thông báo rời mạng. Nếu tình trạng này kéo dài, máy chủ sẽ lưu giữ các địa chỉ mạng ảo, gây lỗi kết nối diện rộng. Để giải quyết, đồ án ứng dụng kỹ thuật nhịp tim kỹ thuật số (Heartbeat). Tại mỗi nút mạng, phương thức báo cáo nhịp tim được đặt trong một vòng lặp vô tận có xen kẽ lệnh tạm nghỉ mười giây, liên tục đẩy gói tin xác nhận sinh tồn lên máy chủ. Phối hợp với đó, máy chủ dẫn đường vận hành một luồng dọn dẹp quét qua toàn bộ từ điển danh bạ mỗi năm giây một lần. Bằng việc tính toán hiệu số giữa thời gian hiện tại của hệ thống và dấu thời gian lưu trong bản ghi của từng người dùng, máy chủ sẽ chủ động loại bỏ bất kỳ thiết bị nào có độ trễ cập nhật nhịp tim vượt quá ngưỡng mười lăm giây, qua đó bảo chứng tính toàn vẹn của đồ thị định tuyến.

3.3.2. Truyền tin cậy và xử lý sự cố ngoại tuyến

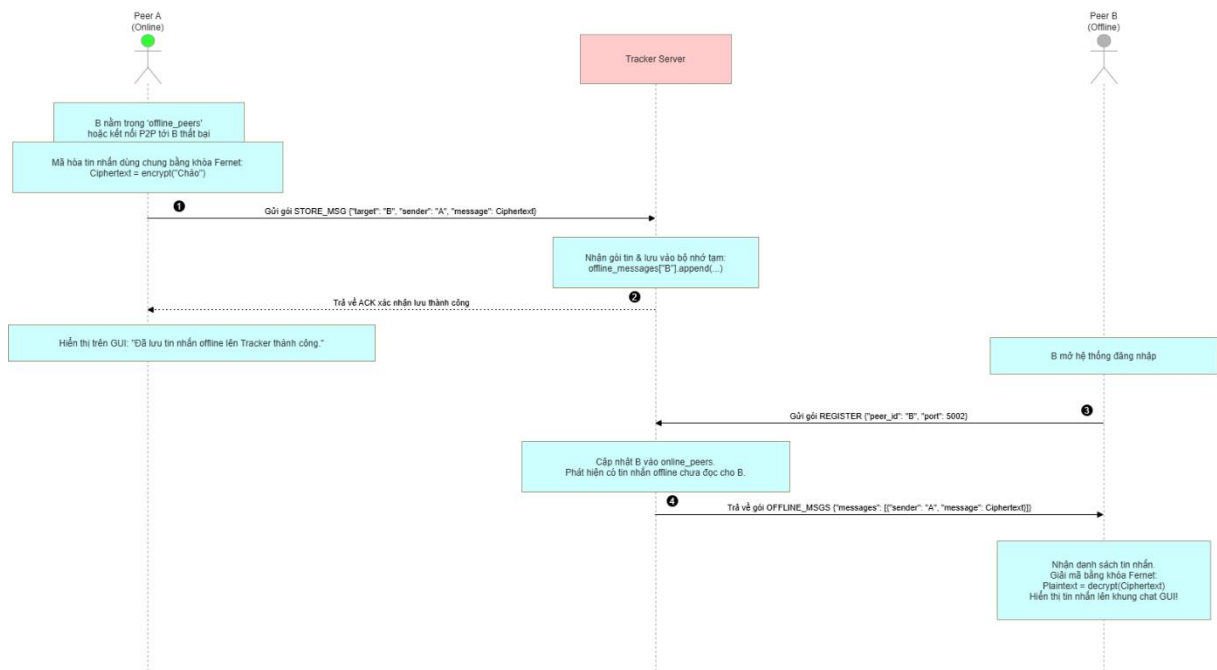
Dữ liệu nhắn tin trong môi trường thực tế luôn phải đối mặt với nguy cơ tắc nghẽn hoặc đứt gãy kết nối mạng. Để đảm bảo tính bền vững, mã nguồn thiết lập cấu hình thời gian chờ tối đa cho các Socket giao tiếp trực tiếp là ba giây. Việc giới hạn thời gian này

giúp hệ thống không bị mắc kẹt vĩnh viễn vào các lời gọi hàm phân bổ dữ liệu khi mạng gặp sự cố.

Khi gửi tin nhắn, tiến trình thực thi của Python sẽ trải qua khối lệnh xử lý ngoại lệ nghiêm ngặt. Nếu thiết bị đích đã chủ động đóng dịch vụ nhưng máy chủ chưa kịp cập nhật, thủ tục thiết lập liên kết ở tầng giao vận sẽ ngay lập tức bị hệ điều hành từ chối, sinh ra ngoại lệ lỗi kết nối. Trong một diễn biến khác, nếu kết nối thành công nhưng đường truyền quá tải khiến thiết bị đích không kịp trả về gói tin xác nhận báo nhận, hệ thống sẽ sinh ra ngoại lệ vượt quá thời gian chờ.

Mỗi khi hệ thống nhận diện được một trong hai ngoại lệ mạng kể trên, thiết bị gửi sẽ tự động hủy bỏ luồng kết nối ngang hàng trực tiếp đang lỗi, đồng thời tái định dạng thông điệp thành gói tin lưu trữ ngoại tuyến. Gói tin này lập tức được chuyển hướng kết nối lên máy chủ dẫn đường. Tại đây, máy chủ duy trì một từ điển riêng biệt để làm hộp thư trung chuyển. Thông điệp được xếp vào danh sách chờ dưới tên của người nhận mục tiêu. Ngay khi phần mềm của người nhận được bật lên và thực thi thành công phương thức đăng ký trạng thái, máy chủ sẽ gom toàn bộ mảng dữ liệu trong hộp thư, đóng gói vào một thông điệp phản hồi đặc biệt và đẩy ngược về thiết bị để hiển thị lên màn hình, triệt tiêu hoàn toàn rủi ro thất thoát thông điệp do đứt gãy hạ tầng.

Để làm rõ hơn tính logic và sự phối hợp đồng bộ giữa các thành phần trong cơ chế lưu trữ và chuyển tiếp này, sơ đồ tuần tự dưới đây sẽ minh họa luồng xử lý chi tiết từ thời điểm phát sinh lỗi định tuyến đến khi thông điệp được khôi phục thành công.



Hình 3.3: Sơ đồ tuần tự cơ chế chuyển tiếp Offline

Dựa theo sơ đồ tuần tự nêu trên, vòng đời của một thông điệp ngoại tuyến được diễn giải qua hai giai đoạn chính:

- Giai đoạn thứ nhất là tiến trình lưu trữ tạm thời. Khi phần mềm tại nút gửi nhận thấy đối tác nằm trong danh bạ ngoại tuyến hoặc gặp sự cố từ chối kết nối, nó sẽ lập tức băm nội dung tin nhắn bằng thuật toán mã hóa đối xứng. Bản mã này sau đó được bọc trong một gói tin mang nhãn điều khiển lưu trữ ngoại tuyến và phóng lên máy chủ dẫn đường. Máy chủ tiếp nhận gói tin, phân loại theo định danh người nhận và bổ sung vào không gian lưu trữ bộ nhớ trong, đồng thời gửi lại tín hiệu xác nhận để thiết bị nguồn in thông báo thành công lên màn hình giao diện.
- Giai đoạn thứ hai là tiến trình khôi phục và hoàn nguyên. Khi người dùng đích khởi động lại phần mềm và phát đi gói tin đăng ký để hòa mạng, máy chủ dẫn đường sẽ ghi nhận trạng thái hoạt động mới. Ngay tại thời điểm đó, máy chủ tiến hành rà soát danh mục hộp thư trung chuyển; nếu phát hiện có dữ liệu tồn đọng dành cho người dùng này, máy chủ sẽ chủ động đóng gói tất cả thành một kiện hàng dữ liệu lớn và đẩy về nút đích. Tại điểm kết thúc, phần mềm tiếp nhận sẽ lần lượt bóc tách từng thông điệp, vận dụng khóa bí mật để dịch ngược bản mã thành văn bản gốc và kết xuất lên khung trò chuyện, khép lại chu trình giao tiếp tin cậy trọn vẹn.

3.3.3. Bảo mật dữ liệu bằng mã hóa đầu cuối

Bảo đảm quyền riêng tư trên một kiến trúc phi tập trung là nhiệm vụ tiên quyết của đồ án. Hệ thống cam kết loại trừ khả năng bị đọc trộm thông điệp, kể cả từ phía người quản trị máy chủ dẫn đường. Về mặt công nghệ, đồ án áp dụng bộ mô-đun Fernet thuộc thư viện mật mã học tiêu chuẩn của ngôn ngữ Python để triển khai cơ chế mã hóa đầu cuối.

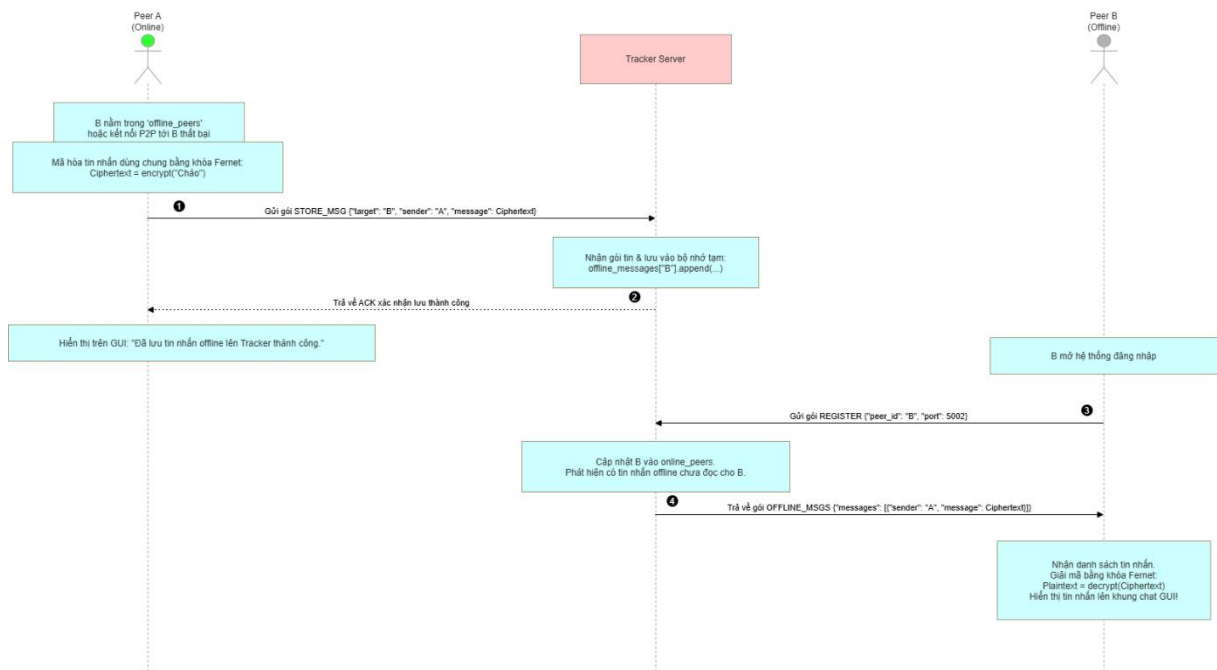
Khung bảo mật này cung cấp một lá chắn kép. Lớp thứ nhất sử dụng thuật toán mã hóa tiêu chuẩn nâng cao (E2EE) với độ dài khóa tĩnh một trăm hai mươi tám bit hoạt động theo cơ chế liên kết chuỗi khối, đảm bảo cùng một nội dung văn bản nhưng khi mã hóa nhiều lần sẽ sinh ra các bản mã hoàn toàn khác nhau. Lớp thứ hai áp dụng kỹ thuật mã xác thực thông điệp sinh ra từ thuật toán băm nhằm ký điện tử vào đuôi của bản mã.

Quy trình bảo mật diễn ra minh bạch. Tại thiết bị gửi, văn bản gốc định dạng chuỗi ký tự được chuyển sang luồng byte, nạp qua hàm mã hóa của bộ thư viện để sinh ra bản mã an toàn trước khi đóng gói vào thông điệp JSON. Khi di chuyển qua mạng hoặc nằm lưu trữ tạm thời trên máy chủ dẫn đường, luồng dữ liệu chỉ hiển thị dưới dạng một chuỗi vô nghĩa. Chỉ khi gói tin hạ cánh tại thiết bị nhận hợp lệ có chứa mã khóa hệ thống, hàm giải mã mới được gọi để kiểm tra tính nguyên vẹn của chữ ký điện tử, và cuối cùng hoàn nguyên luồng byte trở lại thành văn bản có thể đọc được trên giao diện.

3.3.4. Truyền tải tệp tin đa phương tiện

Việc hỗ trợ trao đổi tệp tin dung lượng vừa và nhỏ được hiện thực hóa thông qua một giao thức bắt tay ba bước chuyên biệt kết hợp cùng các phương thức thao tác tệp tin vật lý của hệ điều hành. Giao thức này phân định rõ ràng các chốt chặn kiểm soát nhằm trao quyền quyết định tuyệt đối cho phía thiết bị tiếp nhận, ngăn chặn nguy cơ lây nhiễm mã độc hoặc thư rác.

Để minh họa trực quan luồng giao tiếp bảo mật này, cấu trúc tương tác giữa nút gửi và nút nhận được trình bày chi tiết trong sơ đồ tuần tự dưới đây.



Hình 3.4: Sơ đồ tuần tự truyền tệp tin ba bước

Dựa theo sơ đồ tuần tự, quy trình được khởi tạo khi nút gửi tiến hành chọn một tệp tin cục bộ từ đĩa cứng. Tại thời điểm này, phần mềm chưa lập tức vận chuyển dữ liệu mà chỉ lưu trữ tạm thời đường dẫn vật lý vào không gian bộ nhớ nội bộ.

Bước thứ nhất mang tên yêu cầu truyền tệp. Nút gửi trích xuất các siêu dữ liệu bao gồm tên và kích thước tệp, đóng gói chúng vào một lệnh yêu cầu rồi thiết lập đường truyền TCP gửi thẳng sang nút đích. Ngay sau khi phát đi gói tin này, nút gửi chủ động đóng Socket và chuyển sang trạng thái chờ đợi.

Tại nút đích, luồng mạng ngầm tiếp nhận yêu cầu và đẩy tín hiệu sang cấu trúc hàng đợi. Luồng giao diện chính trích xuất tín hiệu này và lập tức hiển thị một hộp thoại cảnh báo để xin cấp quyền từ người dùng. Từ đây, luồng hệ thống rẽ nhánh theo hai kịch bản độc lập.

Kịch bản thứ nhất diễn ra khi người dùng nút đích đồng ý nhận tệp. Nút đích thực thi bước thứ hai bằng cách mở một liên kết ngược, gửi trả một gói tin xác nhận mang cờ hiệu chấp thuận. Nhận được tín hiệu mở đường này, nút gửi lập tức chuyển sang bước thứ ba. Nó khởi chạy một luồng phụ chuyên trách, mở tệp tin vật lý để đọc các luồng byte nhị phân thô, sau đó áp dụng thuật toán biến đổi cơ số sáu mươi tư nhằm mã hóa toàn bộ cấu trúc nhị phân sang một chuỗi ký tự văn bản an toàn. Chuỗi dữ liệu khổng lồ

này được đóng gói và vận chuyển qua một đường truyền mới với thời gian chờ được nói lỏng. Khi nút đích nhận được, nó tiến hành giải mã văn bản ngược về mảng byte gốc, tạo lập tệp tin mới với tiền tố đánh dấu nhận tệp, ghi dữ liệu xuống đĩa cứng, in thông báo thành công và cuối cùng gửi trả một gói tin báo nhận để khép lại chu trình.

Kịch bản thứ hai phát sinh trong trường hợp người dùng nút đích cảm thấy nghi ngờ và bấm nút từ chối. Bước thứ hai lúc này sẽ mang theo một gói tin phản hồi cự tuyệt. Khi nút gửi tiếp nhận tín hiệu từ chối này, mọi tiến trình đọc tệp và mã hóa dữ liệu đều bị hủy bỏ ngay lập tức, đồng thời giao diện nút gửi sẽ in ra thông báo tệp tin đã bị chặn, qua đó tiết kiệm băng thông và bảo vệ an toàn tuyệt đối cho hệ thống đích.

3.4. Thiết kế kiến trúc giao diện người dùng đa luồng

Thách thức kỹ thuật lớn nhất khi phát triển các phần mềm mạng có tích hợp giao diện người dùng đồ họa là việc giải quyết bài toán tắc nghẽn tiến trình. Thư viện giao diện Tkinter của Python hoạt động dựa trên một vòng lặp sự kiện liên tục và hoàn toàn không hỗ trợ cơ chế an toàn luồng. Nếu các lệnh chờ nhận dữ liệu từ Socket được thực thi trực tiếp trên luồng giao diện chính, toàn bộ phần mềm sẽ rơi vào trạng thái đóng băng, không thể phản hồi bất kỳ tương tác nào từ bàn phím hay chuột của người dùng. Để khắc phục triệt để hiện tượng này, đồ án đã triển khai một kiến trúc phần mềm tuân thủ nghiêm ngặt mô hình Producer - Consumer.

Cô lập xử lý mạng bằng đa luồng chạy nền toàn bộ các nghiệp vụ liên quan đến giao thức mạng đều được bóc tách và tổng khứ khỏi luồng giao diện chính. Ngay khi người dùng hoàn tất thủ tục đăng nhập, phần mềm lập tức khởi tạo hàng loạt các luồng ngầm chuyên biệt. Các luồng này lần lượt đảm nhận các trọng trách độc lập như duy trì Socket lắng nghe liên tục để đón tin nhắn P2P, định kỳ phát tín hiệu nhịp tim lên máy chủ dẫn đường, và liên tục làm mới danh bạ trực tuyến. Nhờ cơ chế này, công việc xử lý dữ liệu nặng ở tầng giao vận được tiến hành hoàn toàn ở lớp nền, giúp vòng lặp sự kiện đồ họa được giải phóng để duy trì tốc độ phản hồi mượt mà nhất.

Cầu nối an toàn thông qua cấu trúc hàng đợi để giải quyết bài toán giao tiếp chéo giữa các luồng mạng và luồng giao diện mà không vi phạm nguyên tắc bảo vệ bộ nhớ, một đối tượng hàng đợi an toàn luồng được khởi tạo làm vùng đệm trung chuyển. Trong mô hình này, các luồng xử lý mạng đóng vai trò là Producer. Bất cứ khi nào thu nhận được

một thông điệp văn bản hoặc một tín hiệu yêu cầu nhận tệp tin từ đối tác, các luồng mạng sẽ tuyệt đối không được phép thao tác trực tiếp lên các biến điều khiển màn hình. Thay vào đó, chúng tiến hành đóng gói nội dung và đẩy thẳng vào bộ đệm Hàng đợi này.

Cơ chế hỏi vòng định kỳ trên giao diện luồng giao diện chính lúc này sẽ sắm vai trò là Consumer thông qua một cơ chế xử lý Hỏi vòng tự động. Bằng việc tận dụng hàm đếm thời gian trễ của thư viện Tkinter, một vòng lặp kiểm tra được thiết lập để tự động kích hoạt lại chu kỳ mỗi một trăm mili-giây. Tại mỗi chu kỳ đánh thức, phần mềm sẽ tiến hành rà soát cấu trúc hàng đợi. Nếu phát hiện có dữ liệu mới tồn đọng, luồng giao diện sẽ rút dữ liệu ra, xử lý phân loại và thực hiện thao tác cập nhật nét vẽ an toàn lên khung màn hình trò chuyện hoặc xuất hiện hộp thoại xác nhận nhận tệp tin.

Quy trình dọn dẹp và giải phóng tài nguyên sự toàn vẹn của hệ thống phân tán phụ thuộc rất nhiều vào khả năng rời mạng văn minh của các thiết bị. Để đảm bảo nút mạng không để lại địa chỉ rác trên máy chủ dẫn đường khi người dùng nhấn nút tắt ứng dụng ở góc cửa sổ, đồ án đã can thiệp vào giao thức đóng cửa sổ cấp hệ thống. Một hàm xử lý sự kiện ngoại lệ được đăng ký lắng nghe hành vi tắt phần mềm. Khi phát hiện người dùng thoát ứng dụng, hàm này lập tức chặn lệnh tắt ngầm, nhanh chóng phát một gói tin điều khiển báo cáo rời mạng lên máy chủ dẫn đường, chờ xác nhận hoàn tất rồi mới tiến hành phá hủy toàn bộ cấu trúc đồ họa và giải phóng hoàn toàn bộ nhớ RAM.

3.5. Tổng kết chương

Chương III đã cung cấp một bản phân tích chuyên sâu và toàn diện về toàn bộ vòng đời xây dựng cấu trúc mã nguồn của hệ thống phần mềm P2P Chat. Vượt qua những rào cản lý thuyết cơ bản, đồ án đã hiện thực hóa thành công một cấu trúc liên kết mạng linh hoạt, trong đó Máy chủ dẫn đường giải quyết triệt để khâu định tuyến ban đầu, nhường lại không gian băng thông khổng lồ cho các thiết bị cá nhân tự do trao đổi dữ liệu trực tiếp.

Các khuyết điểm cố hữu của giao thức TCP đã được thuần hóa thông qua việc tự thiết kế định dạng gói tin lớp ứng dụng kết hợp tuần tự hóa dữ liệu JSON chuẩn xác. Bằng việc ứng dụng thuật toán khóa mật mã đối xứng một trăm hai mươi tám bit cho tin nhắn văn bản và biến đổi cơ sở sáu mươi tư cho luồng tệp tin nhị phân, hệ thống đảm bảo một

môi trường trao đổi an toàn, ẩn danh và chống lại mọi nỗ lực can thiệp từ bên ngoài. Song hành cùng đó, việc ứng phó với sự cố rò rỉ mạng cục bộ thông qua cơ chế lưu trữ ngoại tuyến trên máy chủ đã chứng minh khả năng chịu lỗi và tính linh hoạt cao của hệ thống. Cuối cùng, giải pháp bóc tách luồng giao diện thông qua hàng đợi và cơ chế hồi vòng định kỳ đã mang đến một ứng dụng với độ mượt mà cao, đáp ứng đầy đủ tiêu chí của một phần mềm thương mại hoàn chỉnh. Những kết quả đạt được về mặt kỹ thuật lập trình trong chương này chính là tiền đề vững chắc nhất để hệ thống chính thức bước vào quá trình vận hành mô phỏng và đánh giá thực nghiệm tại Chương IV.

CHƯƠNG IV: THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG

4.1. Môi trường thực nghiệm

Sau quá trình nghiên cứu cơ sở lý thuyết và tiến hành xây dựng kiến trúc mã nguồn, chương này sẽ trình bày các kịch bản thực nghiệm nhằm kiểm chứng tính đúng đắn, độ ổn định và hiệu năng của hệ thống chat P2P. Dựa trên các kết quả thu được, đề án sẽ đưa ra những đánh giá khách quan về mức độ hoàn thành mục tiêu ban đầu, đồng thời đề xuất các định hướng phát triển nâng cao trong tương lai.

Để quá trình kiểm thử phần mềm diễn ra khách quan và có thể tái lập được kết quả, hệ thống được triển khai và đánh giá trên môi trường thực nghiệm với các thông số thiết lập cụ thể như sau:

- Cấu hình phần cứng:
 - Bộ vi xử lý (CPU): Đa nhân, hiệu năng tương đương Intel Core i5 trở lên để xử lý đa tiến trình.
 - Bộ nhớ trong (RAM): Tối thiểu 8GB.
 - Lưu trữ: Ổ cứng SSD nhằm tối ưu tốc độ đọc/ghi khi thử nghiệm truyền tệp tin.
- Môi trường phần mềm:
 - Hệ điều hành: Windows 64-bit.
 - Trình thông dịch: Python phiên bản 3.11.
- Thiết lập mạng:
 - Môi trường: Mạng cục bộ ảo với địa chỉ IP tĩnh 127.0.0.1.
 - Cổng giao tiếp: Máy chủ Tracker Server được gán cố định tại cổng 9000. Các Peer Node được cấp phát cổng linh hoạt từ dải 5000 trở lên để tránh xung đột.
- Thư viện phụ thuộc:
 - Thư viện tiêu chuẩn: socket, threading, queue, json, base64, tkinter, os.

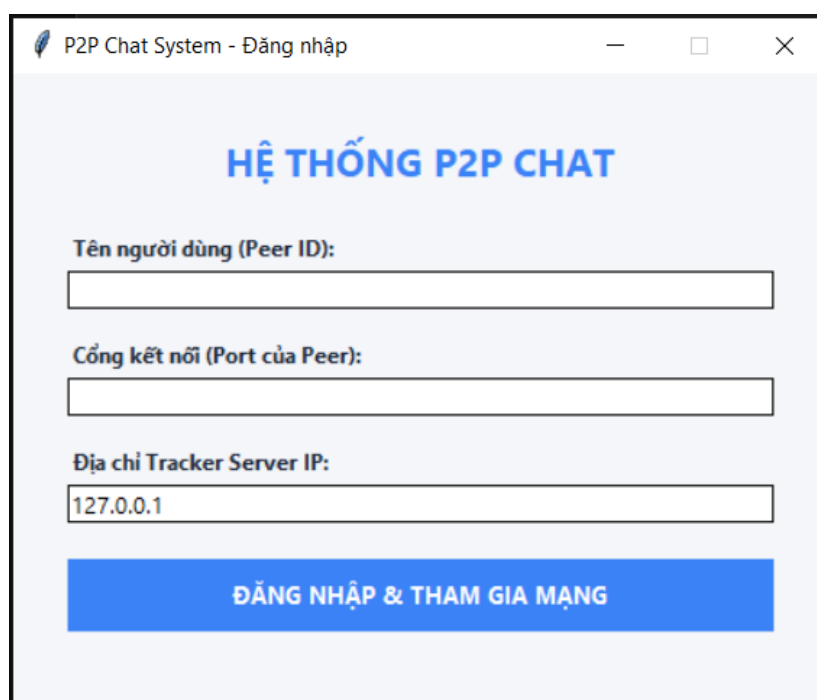
- Thư viện mở rộng: Gói cryptography cung cấp thuật toán mã hóa đối xứng Fernet.

4.2. Kịch bản thực nghiệm chức năng

Quá trình thực nghiệm chức năng được tiến hành thông qua việc thiết lập một không gian mạng khép kín, bao gồm việc khởi chạy một tiến trình máy chủ dẫn đường đóng vai trò điều phối trung tâm và nhiều tiến trình phần mềm máy khách độc lập, đại diện cho các thiết bị của người dùng cuối.

4.2.1. Khởi động hệ thống và khám phá mạng

Kịch bản thử nghiệm bắt đầu bằng thao tác khởi chạy tệp tin *tracker_server.py*. Màn hình giao diện dòng lệnh lập tức xuất hiện thông báo xác nhận máy chủ đã chuyển sang trạng thái “Tracker Server đang lắng nghe tại 0.0.0.0:9000”. Ngay sau đó, ba tiến trình độc lập của phần mềm máy khách được kích hoạt song song. Các tiến trình này lần lượt được đăng nhập dưới các định danh là A, B và C, đồng thời được hệ điều hành cấp phát các cổng kết nối ngẫu nhiên khác nhau để tránh xung đột tài nguyên.

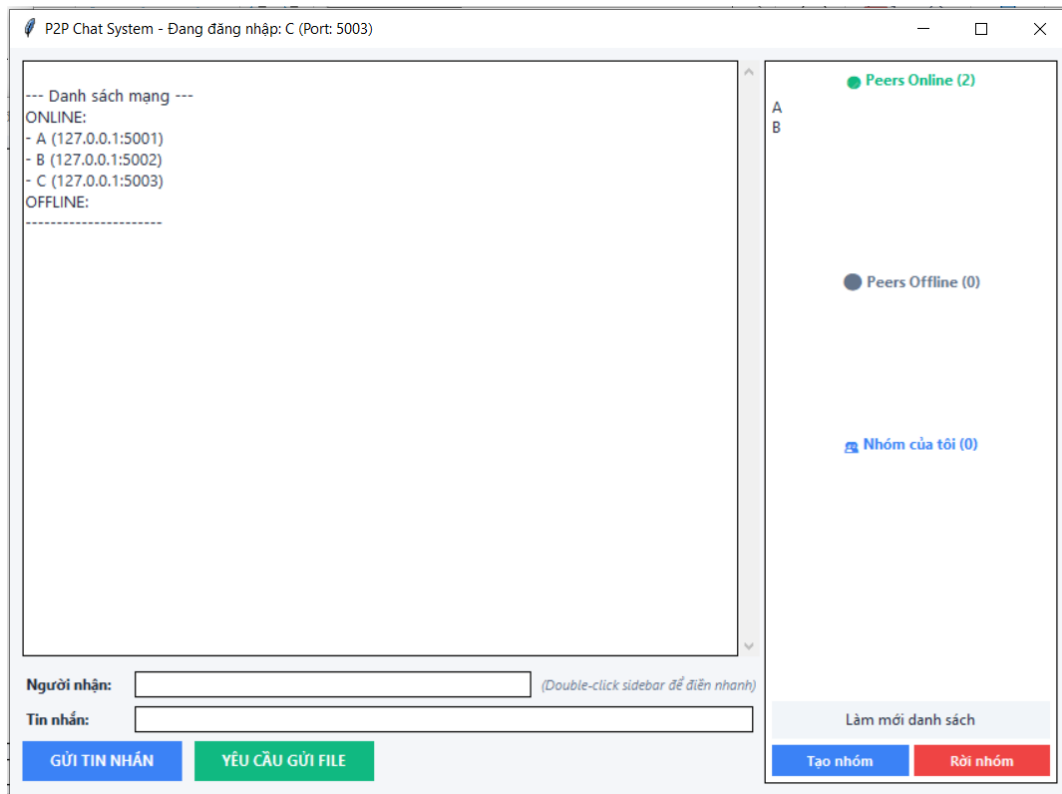


The screenshot shows a web browser window titled "P2P Chat System - Đăng nhập". The main heading is "HỆ THỐNG P2P CHAT" in blue. Below it are three input fields: "Tên người dùng (Peer ID):", "Cổng kết nối (Port của Peer):", and "Địa chỉ Tracker Server IP:". The IP field contains the text "127.0.0.1". At the bottom is a large blue button with the text "ĐĂNG NHẬP & THAM GIA MẠNG".

Hình 4.1: Giao diện đăng nhập hệ thống

Tại khoảnh khắc người dùng nhấn nút “Đăng nhập & tham gia mạng”, hệ thống lập tức ghi nhận các luồng yêu cầu truy cập đổ về máy chủ dẫn đường. Trên giao diện đồ

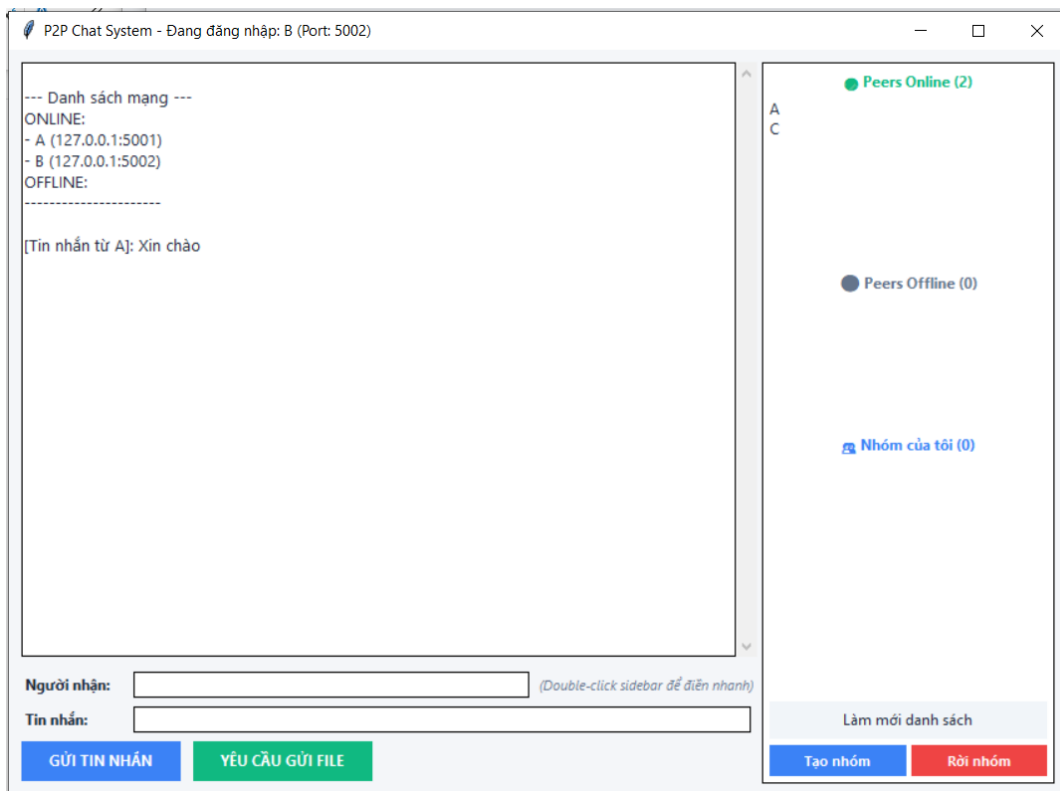
họa của các máy khách, khu vực danh bạ trực tuyến nằm ở thanh điều hướng bên phải tự động cập nhật và hiển thị đầy đủ danh sách các thành viên đang hoạt động trong mạng. Quá trình khám phá mạng này diễn ra mượt mà, phản hồi tức thời, qua đó xác nhận cơ chế đăng ký định danh và thuật toán hỏi vòng cập nhật danh bạ đang hoạt động hoàn toàn chính xác theo đúng thiết kế lý thuyết đề ra.



Hình 4.2: Giao diện chính của hệ thống

4.2.2. Giao tiếp Trực tiếp và nhắn tin nhóm

Để kiểm chứng năng lực giao tiếp mạng ngang hàng cốt lõi, một thông điệp văn bản được khởi tạo từ thiết bị của A với đích đến là định danh của B. Ngay khi lệnh gửi được phát động, màn hình giao diện của B lập tức tiếp nhận và hiển thị thông điệp cùng thời gian nhận thực tế. Một điểm đặc biệt quan trọng trong kịch bản này là việc quan sát bảng điều khiển của máy chủ dẫn đường. Kết quả cho thấy hoàn toàn không có bất kỳ dấu vết nào của gói tin chứa nội dung thông điệp xuất hiện tại đây. Điểm này là minh chứng đắt giá khẳng định kết nối truyền tải dữ liệu đã thực sự được thiết lập song phương và trực tiếp giữa hai thiết bị đầu cuối mà không bị phụ thuộc vào trạm trung chuyển.



Hình 4.3: B nhận được tin nhắn từ A

```
[+] Kết nối mới từ ('127.0.0.1', 50575)
[+] Kết nối mới từ ('127.0.0.1', 50576)
[+] Kết nối mới từ ('127.0.0.1', 50577)
[+] Kết nối mới từ ('127.0.0.1', 50578)
[+] Kết nối mới từ ('127.0.0.1', 50579)
[+] Kết nối mới từ ('127.0.0.1', 50580)
[+] Kết nối mới từ ('127.0.0.1', 50581)
```

Hình 4.4: Không có gói tin chứa nội dung tin nhắn trên Tracker Server

Tiếp nối thành công đó, tính năng trò chuyện tập thể được đưa vào kiểm thử thông qua giao diện tạo phòng thảo luận. A tiến hành tạo một không gian trò chuyện mới bao gồm sự tham gia của cả B và C. Lệnh tạo nhóm được máy chủ dẫn đường tiếp nhận và xác nhận tính hợp lệ. Khi A gửi một tin nhắn vào nhóm này, thuật toán định tuyến đa luồng bắt đầu hoạt động, tự động nhân bản thông điệp và gửi đi song song đến địa chỉ vật lý của cả hai thành viên còn lại. Kết quả thực nghiệm cho thấy khung trò chuyện của B và C đều nhận được nội dung tin nhắn được gắn thẻ tên nhóm một cách đồng thời, chứng minh khả năng xử lý định tuyến đa luồng đồng bộ xuất sắc của hệ thống.

Tạo nhóm mới

Tên nhóm (Group ID):

Group Test

Các thành viên (cách nhau dấu phẩy):

B, C

XÁC NHẬN TẠO NHÓM

Hình 4.5: Giao diện tạo nhóm

P2P Chat System - Đang đăng nhập: A (Port: 5001)

Danh sách mạng ---

ONLINE:

- A (127.0.0.1:5001)

OFFLINE:

[Tôi -> B]: Xin chào

[+] Đã gửi trực tiếp tới B.

[Tôi -> B]: Hello

[+] Đã gửi trực tiếp tới B.

[Hệ thống] Đã gửi yêu cầu tạo nhóm 'Group Test'...

Peers Online (2)

B

C

Peers Offline (0)

Nhóm của tôi (1)

Group Test

Làm mới danh sách

Tạo nhóm

Rời nhóm

Người nhận:

C, A, B

(Double-click sidebar để diển nhanh)

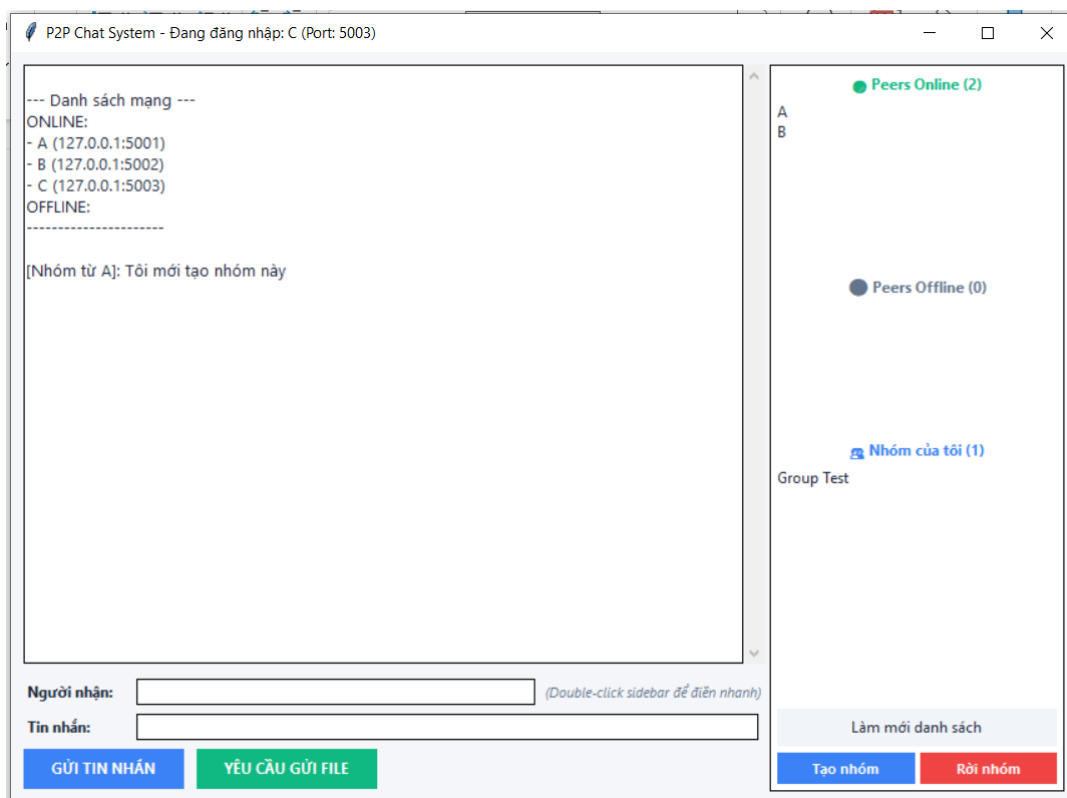
Tin nhắn:

GỬI TIN NHẮN

YÊU CẦU GỬI FILE

Hình 4.6: Khi nhấn vào nhóm muốn gửi tin nhắn, tên người nhận được tự động điền

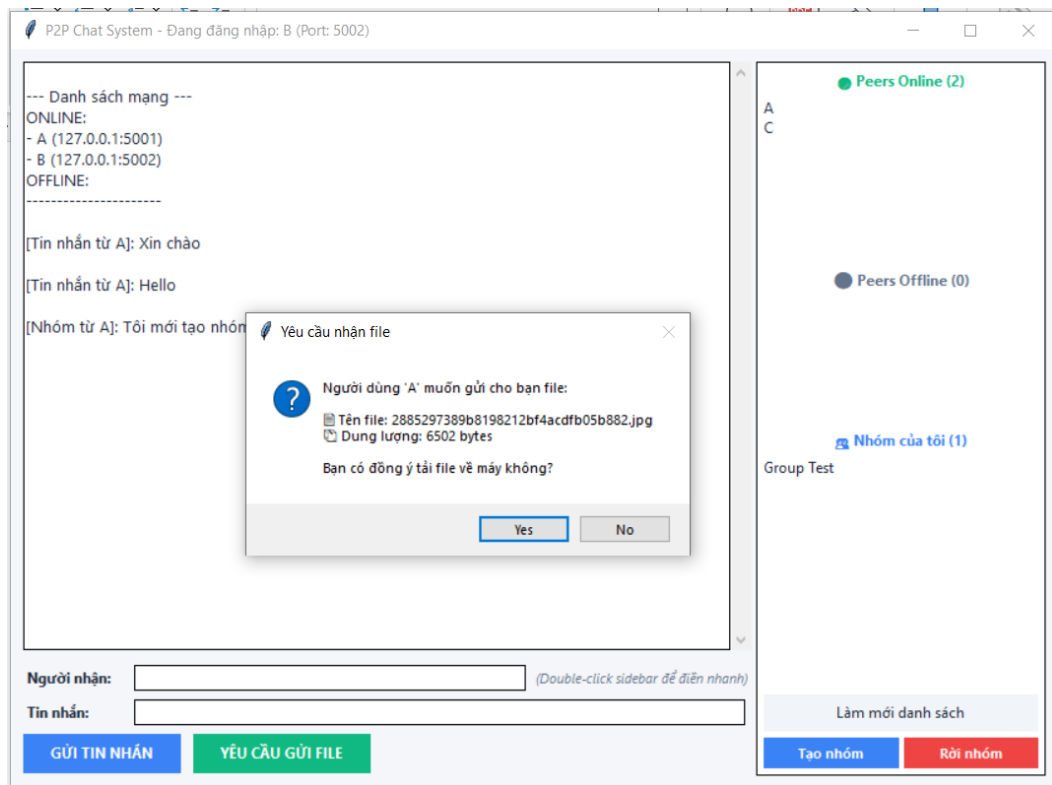
38



Hình 4.7: Tin nhắn nhóm từ A hiển thị trên giao diện của C

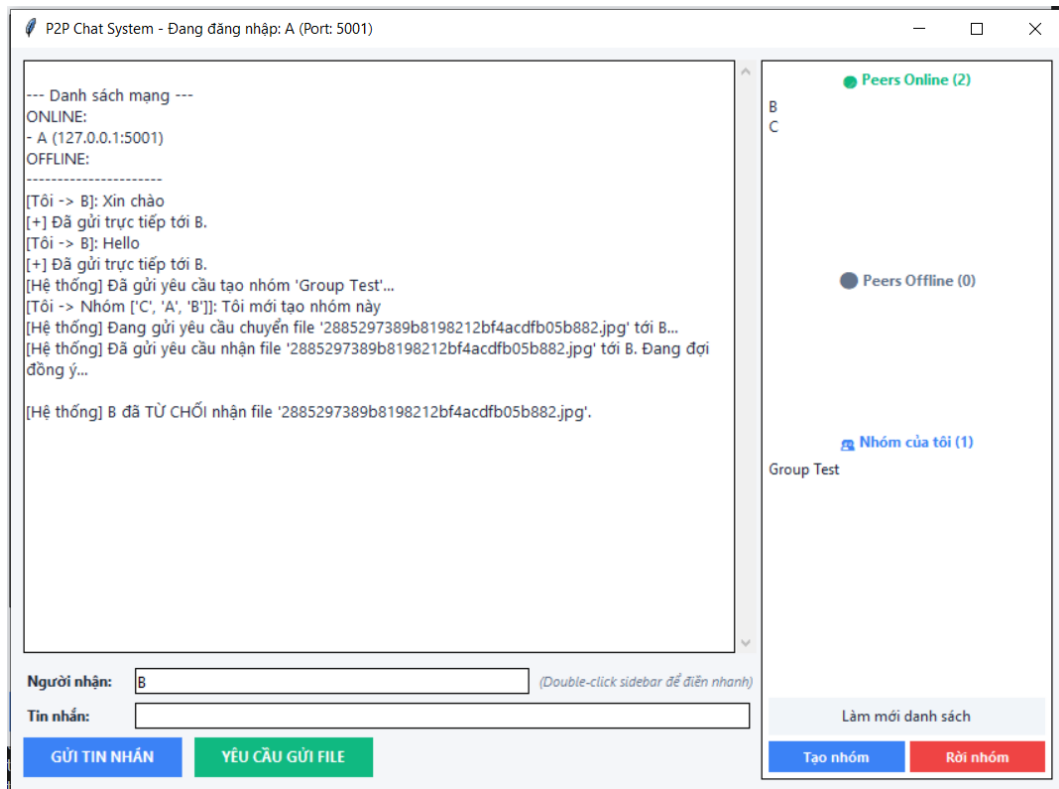
4.2.3. Thực nghiệm Truyền tải tệp tin

Chức năng truyền tệp tin nhị phân được đánh giá một cách khắt khe thông qua quy trình bảo mật ba bước đã được tinh chỉnh. Kích bản bắt đầu khi A ấn nút gửi tệp và lựa chọn một tệp tin hình ảnh lưu trữ cục bộ hướng đến đích là B. Khi đó, hệ thống phát đi một lệnh yêu cầu chuyển giao. Hệ quả là một hộp thoại cảnh báo bảo mật xuất hiện ở vị trí trung tâm trên màn hình của B, thông báo chi tiết về một tệp tin đang chờ xác nhận kèm theo thông số dung lượng cụ thể.

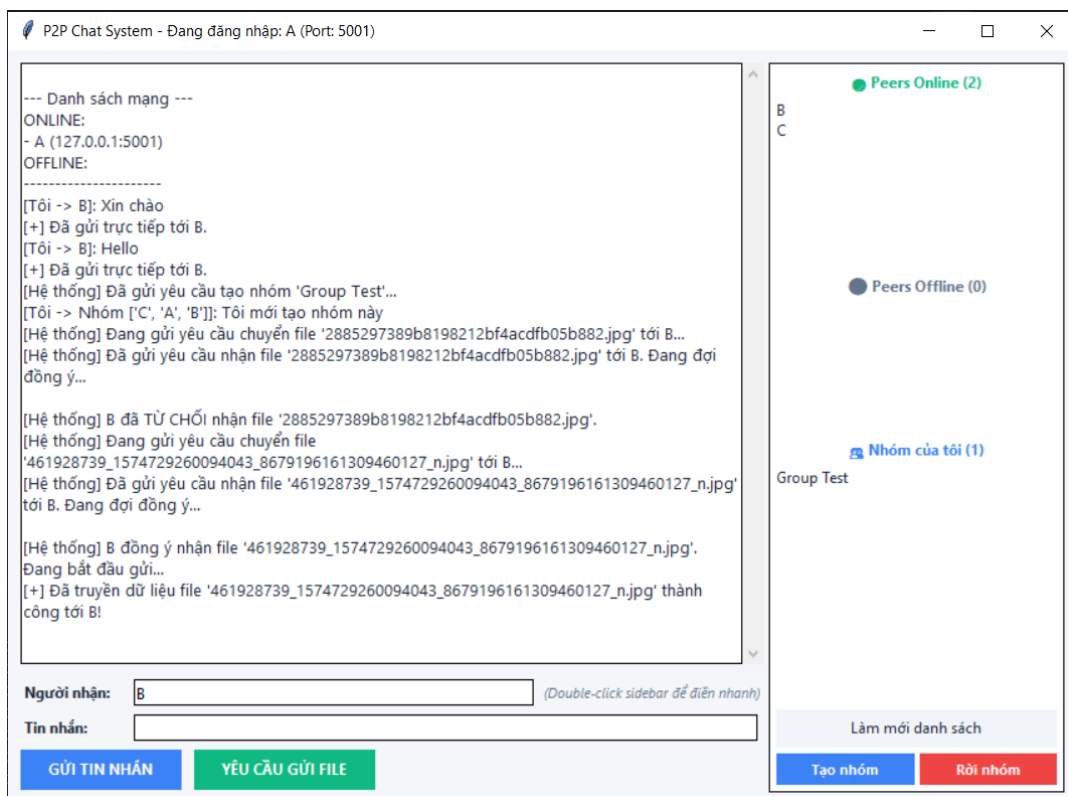


Hình 4.8: Giao diện B xuất hiện thông báo sau khi A gửi file cho B

Ở lần thử nghiệm thứ nhất, khi B nhấn nút từ chối, màn hình của A lập tức phản hồi thông báo tiến trình truyền tải đã bị hủy bỏ bởi người nhận, giúp tiết kiệm băng thông mạng một cách tối ưu. Ở lần thử nghiệm thứ hai, khi B nhấn nút chấp thuận, luồng dữ liệu nhị phân thực sự mới bắt đầu được kích hoạt. Giao diện phần mềm lập tức báo cáo tiến trình tệp tin đang được tải về thiết bị. Sau một khoảng thời gian ngắn chờ đợi, việc kiểm tra thư mục chứa mã nguồn tại thiết bị của B cho thấy tệp tin hình ảnh đã được giải mã và lưu trữ thành công. Người dùng có thể mở tệp tin này lên xem bình thường với chất lượng nguyên bản, chứng minh luồng thuật toán mã hóa và giải mã tuần tự hóa dữ liệu hoạt động hoàn hảo mà không làm suy hao cấu trúc dữ liệu nhị phân nguyên thủy.



Hình 4.9: Ở kịch bản 1, thông báo B từ chối nhận file ở giao diện của A



Hình 4.10: Ở kịch bản 2, thông báo B đồng ý nhận file ở giao diện của A

4.2.4. Thực nghiệm cơ chế lưu trữ và chuyển tiếp ngoại tuyến

Kịch bản kiểm thử độ bền bỉ của hệ thống được tiến hành thông qua việc giả lập tình huống đứt gãy kết nối mạng bằng cách cưỡng chế tắt phần mềm của B. Tại thiết bị của A, người dùng tiếp tục soạn thảo và phát đi một đoạn văn bản hướng đến B. Nhờ cơ chế bắt ngoại lệ đã được lập trình sẵn, thay vì báo lỗi sập ứng dụng do kết nối bị từ chối, giao diện của A tự động chuyển trạng thái và in ra thông báo hệ thống đã kích hoạt tính năng gửi lưu trữ qua trạm trung chuyển.

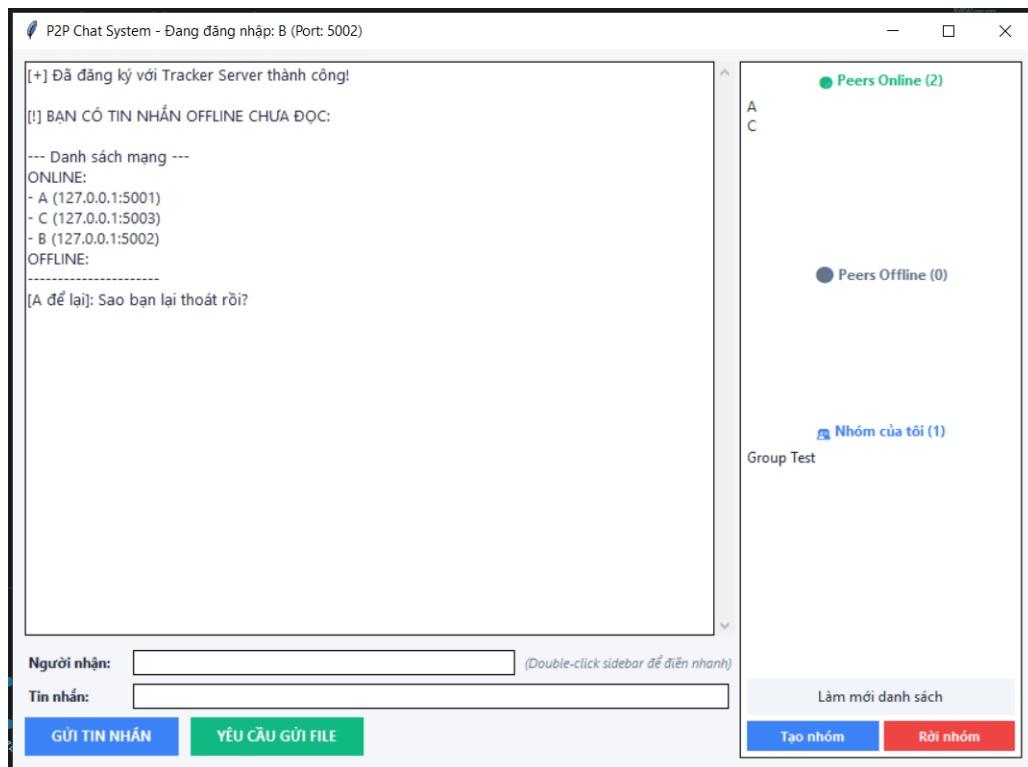
```
[Tôi -> B]: Sao bạn lại thoát rồi?  
[-] Lỗi kết nối trực tiếp đến B. Kích hoạt Store-and-Forward gửi lên Tracker...  
[+] Đã lưu tin nhắn offline lên Tracker Server.
```

Hình 4.11: Tin nhắn tự động được lưu trữ lên Tracker Server

```
[+] Kết nối mới từ ('127.0.0.1', 52846)  
[+] Kết nối mới từ ('127.0.0.1', 52848)  
[*] Đã lưu tạm tin nhắn từ A gửi cho B.  
[+] Kết nối mới từ ('127.0.0.1', 52849)  
[+] Kết nối mới từ ('127.0.0.1', 52851)
```

Hình 4.12: Thông báo lưu tạm tin nhắn trên Tracker Server

Di chuyển sự chú ý sang cửa sổ quản trị của máy chủ dẫn đường, hệ thống in ra thông báo xác nhận đã tiếp nhận thành công một thông điệp lưu trữ tạm thời dành riêng cho B. Để hoàn tất kịch bản, phần mềm của B được khởi động trở lại. Ngay khoảnh khắc B nhấn nút đăng nhập thành công, một thông báo hộp thư đến lập tức xuất hiện trên giao diện chính, ngay theo sau là sự xuất hiện đầy đủ của nội dung đoạn văn bản mà A đã gửi trong lúc B vắng mặt. Kịch bản khép kín này khẳng định tính đúng đắn tuyệt đối của cơ chế xử lý ngoại lệ và độ tin cậy vượt trội của giao thức chuyển tiếp thông điệp ngoại tuyến.



Hình 4.13: Tin nhắn được chuyển đến B ngay khi B online trở lại

4.3. Mô phỏng biến động mạng với Churn Simulator

Một trong những bài toán hóc búa nhất của mạng ngang hàng là khả năng chịu đựng hiện tượng Churn khi các thiết bị gia nhập và rời mạng với tần suất dày đặc. Để kiểm thử giới hạn hệ thống, đồ án đã tự phát triển một trình giả lập biến động mạng độc lập mang tên *churn_simulate.py*.

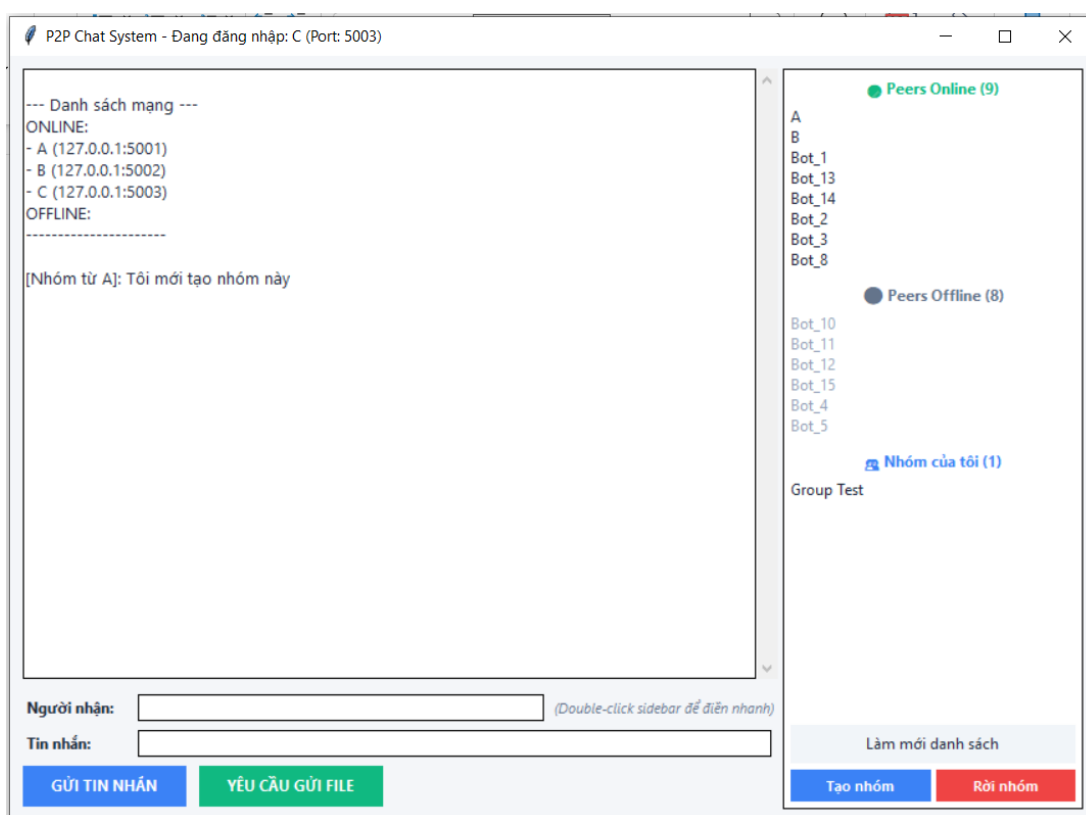
Trình giả lập này thiết lập mười lăm máy khách tự động hoạt động dưới dạng các bot được sinh tự động dựa trên đoạn mã Python. Kịch bản thử nghiệm được cấu hình để tạo ra các khoảng nghỉ ngẫu nhiên từ hai đến năm giây. Hết thời gian nghỉ, một bot ngẫu nhiên sẽ thay đổi trạng thái, thực hiện kết nối hòa mạng nếu đang ngoại tuyến hoặc ngắt kết nối hoàn toàn nếu đang trực tuyến. Song song đó, một luồng ngầm liên tục gửi tín hiệu nhịp tim duy trì kết nối cho các bot đang hoạt động.

```
TRÌNH GIẢ LẬP BIẾN ĐỘNG MẠNG P2P (CHURN SIMULATOR)
=====
[*] Đang kết nối tới Tracker Server tại: 127.0.0.1:9000
[*] Tổng số bot giả lập tự động: 15 bot (Bot_1 đến Bot_15)
[*] Nhấn Ctrl+C để dừng trình giả lập bất kỳ lúc nào.

[ONLINE] Bot_1 (Port: 6001) đã tham gia mạng.
[ONLINE] Bot_2 (Port: 6002) đã tham gia mạng.
[ONLINE] Bot_3 (Port: 6003) đã tham gia mạng.
```

Hình 4.14: Khởi chạy churn_simulate.py

Kết quả thực nghiệm diễn ra cực kỳ khả quan. Máy chủ dẫn đường hứng chịu hàng loạt gói tin đăng ký, rời mạng và nhịp tim đổ về liên tục nhưng không hề gặp hiện tượng tràn bộ nhớ hay lỗi cấp phát tài nguyên. Cơ chế dọn dẹp điểm chết hoạt động trơn tru. Quan sát trên một phần mềm giao diện đồ họa đang mở song song của một người dùng thật, danh sách trực tuyến ở thanh điều hướng liên tục nhấp nháy, tự động làm mới thêm bớt danh tính của các robot theo thời gian thực mà không làm gián đoạn trải nghiệm gõ phím hay làm đứng hình phần mềm. Kết quả này là minh chứng mạnh mẽ nhất cho năng lực xử lý đa luồng và tính bền vững của kiến trúc hệ thống đã lựa chọn.



Hình 4.15: Hệ thống vẫn hoạt động bình thường dù cho nhiều bot ra vào mạng liên tục

4.4. Đánh giá hệ thống

4.4.1. Kết quả đạt được

Các thành tựu cốt lõi đã được hiện thực hóa trong đồ án bao gồm:

- Về kiến trúc và định tuyến: Xây dựng thành công mô hình mạng ngang hàng lai Hybrid P2P. Việc phân tách rõ ràng luồng thông điệp điều khiển thông qua Tracker và luồng dữ liệu trực tiếp giữa các Peer đã loại bỏ hoàn toàn nút thắt cổ chai truyền thống, đảm bảo tốc độ truyền tải đạt mức tối ưu.
- Về tính năng toàn diện: Hệ thống hiện thực hóa thành công các tác vụ nghiệp vụ quan trọng như trò chuyện cá nhân, trò chuyện nhóm đa luồng, hỗ trợ cơ chế xác nhận 3 bước khi truyền file, và đặc biệt là cơ chế "lưu trữ và chuyển tiếp" giúp thông điệp không bị mất mát khi thiết bị đích ngoại tuyến.
- Về bảo mật và trải nghiệm: Tích hợp thành công thư viện mã hóa Fernet cho các thông điệp trao đổi, bảo vệ dữ liệu người dùng khỏi các cuộc tấn công nghe lén trên đường truyền. Mô hình thiết kế đa luồng kết hợp hàng đợi đảm bảo giao diện luôn mượt mà, phản hồi ngay lập tức và không xảy ra hiện tượng treo ứng dụng.
- Về độ ổn định: Thông qua trình mô phỏng Churn, hệ thống đã chứng minh được tính bền vững cao trước các biến động mạng phức tạp, giữ vững khả năng duy trì trạng thái danh bạ mà không gặp lỗi tràn bộ nhớ hoặc mất kết nối cục bộ.

4.4.2. Nhược điểm và hạn chế

Bên cạnh những điểm tích cực, hệ thống vẫn tồn tại các khiếm khuyết kỹ thuật cần khắc phục:

- Sự phụ thuộc vào Tracker Server: Dù đã giảm tải đáng kể, kiến trúc vẫn mang tính "bán phân tán" vì Tracker Server vẫn giữ vai trò điều phối trung tâm. Nếu Server này gặp sự cố, các nút mạng mới không thể khám phá nhau, làm giảm tính tự chủ hoàn toàn của mô hình P2P thuần túy.
- Hiệu năng tệp tin dung lượng lớn: Việc mã hóa Base64 toàn bộ tệp tin trước khi truyền tải tạo ra áp lực rất lớn lên bộ nhớ RAM của thiết bị. Với các file dung

lượng lớn, quá trình này dễ gây ra hiện tượng giật lag hoặc quá tải bộ nhớ trên các máy cấu hình thấp.

- Tính bảo mật của khóa: Khóa mã hóa đang được quản lý theo dạng cấu hình tĩnh, tiềm ẩn rủi ro nếu tệp tin cấu hình bị lộ. Hệ thống hiện chưa có các giao thức trao đổi khóa động để tự động làm mới khóa theo phiên.
- Khả năng xuyên phá mạng: Hệ thống hiện chỉ hoạt động tốt trong phạm vi mạng cục bộ. Việc chưa tích hợp cơ chế đục lỗ NAT khiến các peer nằm sau các lớp tường lửa khác nhau không thể thiết lập kết nối trực tiếp, hạn chế khả năng mở rộng hệ thống ra môi trường Internet công cộng.

4.5. Định hướng phát triển tương lai

Bên cạnh những kết quả tích cực đã đạt được, hệ thống vẫn tồn tại một số điểm giới hạn do rào cản về mặt thời gian và khối lượng của một đồ án môn học. Các giới hạn này mở ra những định hướng phát triển đầy tiềm năng để nâng cấp phần mềm sau này:

- Tối ưu hóa truyền tải tệp tin dung lượng cực lớn: Hiện tại, giải pháp mã hóa Base64 trực tiếp nhúng vào thông điệp cấu trúc bộc lộ điểm yếu khi xử lý các tệp tin có dung lượng hàng GB do yêu cầu nạp toàn bộ vào bộ nhớ chính. Hướng phát triển tới là xây dựng một luồng Socket truyền byte nhị phân riêng biệt, áp dụng thuật toán cắt lát tệp tin để vừa truyền vừa lưu đệm trên đĩa cứng, giúp tiết kiệm tối đa tài nguyên hệ thống.
- Nâng cấp cơ sở hạ tầng khóa bảo mật: Việc sử dụng một khóa bí mật cố định chia sẻ trước là một rủi ro nếu mã nguồn bị lộ. Hệ thống cần được tích hợp thêm giao thức trao đổi khóa Diffie-Hellman hoặc thuật toán mã hóa bất đối xứng RSA. Điều này giúp hai nút mạng có thể tự thỏa thuận và sinh ra một khóa mã hóa đối xứng dùng một lần cho mỗi phiên giao tiếp, nâng cao mức độ bảo mật.
- Triển khai kỹ thuật xuyên phá tường lửa: Để hệ thống có thể hoạt động rộng rãi qua môi trường Internet toàn cầu thay vì giới hạn trong mạng cục bộ, đồ án cần nghiên cứu tích hợp các giao thức phá rào định tuyến, triển khai các máy chủ hỗ trợ vượt rào để đục lỗ tạo đường hầm kết nối trực tiếp cho các thiết bị mạng nội bộ.

LỜI KẾT

Sau một chặng đường nghiên cứu và phát triển đầy thử thách, đồ án "**Phát triển hệ thống chat ngang hàng P2P**" đã hoàn thành mục tiêu đề ra, hiện thực hóa một giải pháp truyền thông phi tập trung với khả năng kết nối linh hoạt và bảo mật cao. Quá trình thực hiện đề tài không chỉ là một bài toán kỹ thuật đơn thuần, mà còn là cơ hội quý báu để tôi tiếp cận sâu sắc hơn với những nguyên lý nền tảng của hệ thống phân tán, kỹ thuật lập trình socket đa luồng và các cơ chế bảo mật thông tin hiện đại.

Mặc dù hệ thống vẫn còn những hạn chế kỹ thuật tất yếu, đặc biệt là trong việc xử lý các tập tin dung lượng lớn hay khả năng vận hành qua mạng Internet công cộng bị ngăn chặn bởi tường lửa, nhưng những kết quả đạt được qua các kịch bản thực nghiệm đã chứng minh tiềm năng to lớn của mô hình mạng ngang hàng lai Hybrid P2P. Việc giải quyết thành công bài toán Churn thông qua trình giả lập biến động mạng hay đảm bảo trải nghiệm người dùng mượt mà với mô hình đa luồng là những cột mốc quan trọng, tạo nền tảng vững chắc cho việc tư duy về các kiến trúc hệ thống quy mô lớn trong tương lai.

Quan trọng hơn cả, đồ án đã mang lại cho tôi cái nhìn thực tế và tư duy phản biện trong việc thiết kế phần mềm. Tôi nhận ra rằng, trong kỹ thuật phần mềm, không bao giờ có một giải pháp vạn năng; mọi thiết kế đều là sự đánh đổi khôn ngoan giữa tính năng, hiệu năng và độ phức tạp. Những kỹ năng giải quyết vấn đề, tinh thần không ngại thử nghiệm và khả năng biến những kiến thức lý thuyết trừu tượng thành một sản phẩm công nghệ có khả năng vận hành trơn tru chính là thành quả đáng trân trọng nhất mà tôi thu nhận được sau quá trình này.

Kết thúc đồ án, tôi xin chân thành cảm ơn những kiến thức và định hướng quý báu từ các giảng viên. Dù hành trình của đồ án môn học có điểm dừng, nhưng những bài học về kiến trúc mạng và tư duy lập trình hệ thống sẽ tiếp tục là hành trang quan trọng cho sự phát triển chuyên môn của tôi trong tương lai. Hy vọng rằng, với những tiền đề đã được thiết lập, hệ thống này sẽ là điểm khởi đầu cho những nghiên cứu sâu rộng hơn, hướng tới việc xây dựng các ứng dụng mạng mạnh mẽ, an toàn và tự chủ hơn nữa.

TÀI LIỆU THAM KHẢO

- Giáo trình Các hệ thống Phân tán 2024 (PGS. TS. Phạm Văn Cường, ThS. Nguyễn Xuân Anh).
- Distributed Systems - Third edition (Maarten van Steen, Andrew S. Tanenbaum).
- A Hybrid P2P Enabled Big Data Resource Discovery Method in Cloud Environments (Wanjing Wu, Liang Tan, Ziyuan Yu, Peng Yang, Danlian Ye, Kun She):

<https://www.sciopen.com/article/10.26599/TST.2025.9010038>

- Mã hóa End-to-end encryption là gì? Cách hoạt động của E2EE (Trịnh Duy Thanh):

<https://bkhost.vn/blog/ma-hoa-end-to-end-encryption-e2ee/>

- Socket Programming in Python (Nathan Jennings):

<https://realpython.com/python-sockets/>

- Multithreading in Python:

<https://www.geeksforgeeks.org/python/multithreading-python-set-1/>

- Fernet (symmetric encryption):

<https://cryptography.io/en/latest/fernet/>

- Python Tkinter:

<https://www.geeksforgeeks.org/python/python-gui-tkinter/>